



Learning Apache Spark with Python

Wenqiang Feng

November 21, 2021

CONTENTS

1	Preface	3
1.1	About	3
1.2	Motivation for this tutorial	4
1.3	Copyright notice and license info	4
1.4	Acknowledgement	5
1.5	Feedback and suggestions	5
2	Why Spark with Python ?	7
2.1	Why Spark?	7
2.2	Why Spark with Python (PySpark)?	8
3	Configure Running Platform	11
3.1	Run on Databricks Community Cloud	11
3.2	Configure Spark on Mac and Ubuntu	16
3.3	Configure Spark on Windows	19
3.4	PySpark With Text Editor or IDE	19
3.5	PySparkling Water: Spark + H2O	26
3.6	Set up Spark on Cloud	27
3.7	PySpark on Colaboratory	28
3.8	Demo Code in this Section	28
4	An Introduction to Apache Spark	31
4.1	Core Concepts	31
4.2	Spark Components	32
4.3	Architecture	34
4.4	How Spark Works?	34
5	Programming with RDDs	35
5.1	Create RDD	35
5.2	Spark Operations	39
5.3	<code>rdd.DataFrame</code> vs <code>pd.DataFrame</code>	41
6	Statistics and Linear Algebra Preliminaries	59
6.1	Notations	59
6.2	Linear Algebra Preliminaries	59
6.3	Measurement Formula	61

6.4	Confusion Matrix	62
6.5	Statistical Tests	63
7	Data Exploration	65
7.1	Univariate Analysis	65
7.2	Multivariate Analysis	79
8	Data Manipulation: Features	87
8.1	Feature Extraction	87
8.2	Feature Transform	97
8.3	Feature Selection	116
8.4	Unbalanced data: Undersampling	117
9	Regression	119
9.1	Linear Regression	119
9.2	Generalized linear regression	133
9.3	Decision tree Regression	142
9.4	Random Forest Regression	150
9.5	Gradient-boosted tree regression	158
10	Regularization	167
10.1	Ordinary least squares regression	167
10.2	Ridge regression	168
10.3	Least Absolute Shrinkage and Selection Operator (LASSO)	168
10.4	Elastic net	168
11	Classification	169
11.1	Binomial logistic regression	169
11.2	Multinomial logistic regression	181
11.3	Decision tree Classification	194
11.4	Random forest Classification	206
11.5	Gradient-boosted tree Classification	216
11.6	XGBoost: Gradient-boosted tree Classification	217
11.7	Naive Bayes Classification	219
12	Clustering	233
12.1	K-Means Model	233
13	RFM Analysis	247
13.1	RFM Analysis Methodology	248
13.2	Demo	250
13.3	Extension	256
14	Text Mining	263
14.1	Text Collection	263
14.2	Text Preprocessing	271
14.3	Text Classification	274
14.4	Sentiment analysis	280
14.5	N-grams and Correlations	287

14.6	Topic Model: Latent Dirichlet Allocation	287
15	Social Network Analysis	305
15.1	Introduction	306
15.2	Co-occurrence Network	306
15.3	Appendix: matrix multiplication in PySpark	311
15.4	Correlation Network	312
16	ALS: Stock Portfolio Recommendations	313
16.1	Recommender systems	314
16.2	Alternating Least Squares	315
16.3	Demo	315
17	Monte Carlo Simulation	323
17.1	Simulating Casino Win	324
17.2	Simulating a Random Walk	324
18	Markov Chain Monte Carlo	335
18.1	Metropolis algorithm	336
18.2	A Toy Example of Metropolis	336
18.3	Demos	337
19	Neural Network	345
19.1	Feedforward Neural Network	345
20	Automation for Cloudera Distribution Hadoop	349
20.1	Automation Pipeline	349
20.2	Data Clean and Manipulation Automation	349
20.3	ML Pipeline Automation	352
20.4	Save and Load PipelineModel	353
20.5	Ingest Results Back into Hadoop	353
21	Wrap PySpark Package	355
21.1	Package Wrapper	355
21.2	Pacakge Publishing on PyPI	357
22	PySpark Data Audit Library	359
22.1	Install with pip	359
22.2	Install from Repo	359
22.3	Uninstall	359
22.4	Test	360
22.5	Auditing on Big Dataset	361
23	Zeppelin to jupyter notebook	371
23.1	How to Install	371
23.2	Converting Demos	372
24	My Cheat Sheet	377

25 JDBC Connection	381
25.1 JDBC Driver	381
25.2 JDBC read	381
25.3 JDBC write	383
25.4 JDBC temp_view	383
26 Databricks Tips	385
26.1 Display samples	385
26.2 Auto files download	385
26.3 delta format	387
26.4 mlflow	387
27 PySpark API	391
27.1 Stat API	391
27.2 Regression API	397
27.3 Classification API	416
27.4 Clustering API	436
27.5 Recommendation API	451
27.6 Pipeline API	456
27.7 Tuning API	458
27.8 Evaluation API	463
28 Main Reference	469
Bibliography	471
Python Module Index	473
Index	475



Welcome to my **Learning Apache Spark with Python** note! In this note, you will learn a wide array of concepts about **PySpark** in Data Mining, Text Mining, Machine Learning and Deep Learning. The PDF version can be downloaded from [HERE](#).

1.1 About

1.1.1 About this note

This is a shared repository for [Learning Apache Spark Notes](#). The PDF version can be downloaded from [HERE](#). The first version was posted on Github in [ChenFeng](#) ([Feng2017]). This shared repository mainly contains the self-learning and self-teaching notes from Wenqiang during his [IMA Data Science Fellowship](#). The reader is referred to the repository <https://github.com/runawayhorse001/LearningApacheSpark> for more details about the `dataset` and the `.ipynb` files.

In this repository, I try to use the detailed demo code and examples to show how to use each main functions. If you find your work wasn't cited in this note, please feel free to let me know.

Although I am by no means an data mining programming and Big Data expert, I decided that it would be useful for me to share what I learned about PySpark programming in the form of easy tutorials with detailed example. I hope those tutorials will be a valuable tool for your studies.

The tutorials assume that the reader has a preliminary knowledge of programming and Linux. And this document is generated automatically by using `sphinx`.

1.1.2 About the author

- **Wenqiang Feng**
 - Director of Data Science and PhD in Mathematics
 - University of Tennessee at Knoxville
 - Email: von198@gmail.com

- **Biography**

Wenqiang Feng is the Director of Data Science at American Express (AMEX). Prior to his time at AMEX, Dr. Feng was a Sr. Data Scientist in Machine Learning Lab, H&R Block. Before joining Block, Dr. Feng was a Data Scientist at Applied Analytics Group, DST (now SS&C). Dr. Feng's responsibilities include providing clients with access to cutting-edge skills and technologies, including Big Data analytic solutions, advanced analytic and data enhancement techniques and modeling.

Dr. Feng has deep analytic expertise in data mining, analytic systems, machine learning algorithms, business intelligence, and applying Big Data tools to strategically solve industry problems in a cross-functional business. Before joining DST, Dr. Feng was an IMA Data Science Fellow at The Institute for Mathematics and its Applications (IMA) at the University of Minnesota. While there, he helped startup companies make marketing decisions based on deep predictive analytics.

Dr. Feng graduated from University of Tennessee, Knoxville, with Ph.D. in Computational Mathematics and Master's degree in Statistics. He also holds Master's degree in Computational Mathematics from Missouri University of Science and Technology (MST) and Master's degree in Applied Mathematics from the University of Science and Technology of China (USTC).

- **Declaration**

The work of Wenqiang Feng was supported by the IMA, while working at IMA. However, any opinion, finding, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the IMA, UTK, DST, HR & Block and AMEX.

1.2 Motivation for this tutorial

I was motivated by the [IMA Data Science Fellowship](#) project to learn PySpark. After that I was impressed and attracted by the PySpark. And I found that:

1. It is no exaggeration to say that Spark is the most powerful Bigdata tool.
2. However, I still found that learning Spark was a difficult process. I have to Google it and identify which one is true. And it was hard to find detailed examples which I can easily learn the full process in one file.
3. Good sources are expensive for a graduate student.

1.3 Copyright notice and license info

This [Learning Apache Spark with Python](#) PDF file is supposed to be a free and living document, which is why its source is available online at <https://runawayhorse001.github.io/LearningApacheSpark/pyspark.pdf>. But this document is licensed according to both [MIT License](#) and [Creative Commons Attribution-NonCommercial 2.0 Generic \(CC BY-NC 2.0\) License](#).

When you plan to use, copy, modify, merge, publish, distribute or sublicense, Please see the terms of those licenses for more details and give the corresponding credits to the author.

1.4 Acknowledgement

At here, I would like to thank Ming Chen, Jian Sun and Zhongbo Li at the University of Tennessee at Knoxville for the valuable discussion and thank the generous anonymous authors for providing the detailed solutions and source code on the internet. Without those help, this repository would not have been possible to be made. Wenqiang also would like to thank the [Institute for Mathematics and Its Applications \(IMA\)](#) at [University of Minnesota, Twin Cities](#) for support during his IMA Data Scientist Fellow visit and thank TAN THIAM HUAT and Mark Rabins for finding the typos.

A special thank you goes to [Dr. Haiping Lu](#), Lecturer in Machine Learning at Department of Computer Science, University of Sheffield, for recommending and heavily using my tutorial in his teaching class and for the valuable suggestions.

1.5 Feedback and suggestions

Your comments and suggestions are highly appreciated. I am more than happy to receive corrections, suggestions or feedbacks through email (von198@gmail.com) for improvements.

WHY SPARK WITH PYTHON ?

Chinese proverb

Sharpening the knife longer can make it easier to hack the firewood – old Chinese proverb

I want to answer this question from the following two parts:

2.1 Why Spark?

I think the following four main reasons from [Apache Spark™](#) official website are good enough to convince you to use Spark.

1. Speed

Run programs up to 100x faster than Hadoop MapReduce in memory, or 10x faster on disk.

Apache Spark has an advanced DAG execution engine that supports acyclic data flow and in-memory computing.

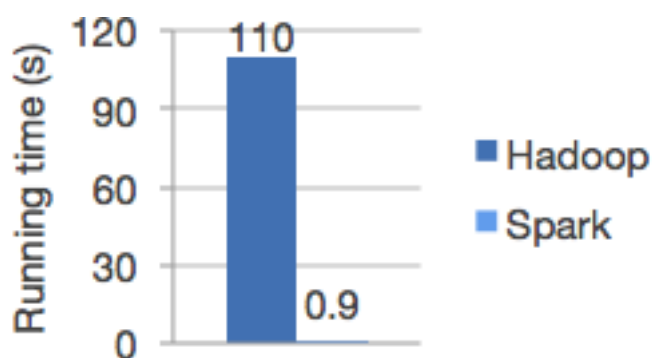


Fig. 1: Logistic regression in Hadoop and Spark

2. Ease of Use

Write applications quickly in Java, Scala, Python, R.

Spark offers over 80 high-level operators that make it easy to build parallel apps. And you can use it interactively from the Scala, Python and R shells.

3. Generality

Combine SQL, streaming, and complex analytics.

Spark powers a stack of libraries including SQL and DataFrames, MLlib for machine learning, GraphX, and Spark Streaming. You can combine these libraries seamlessly in the same application.

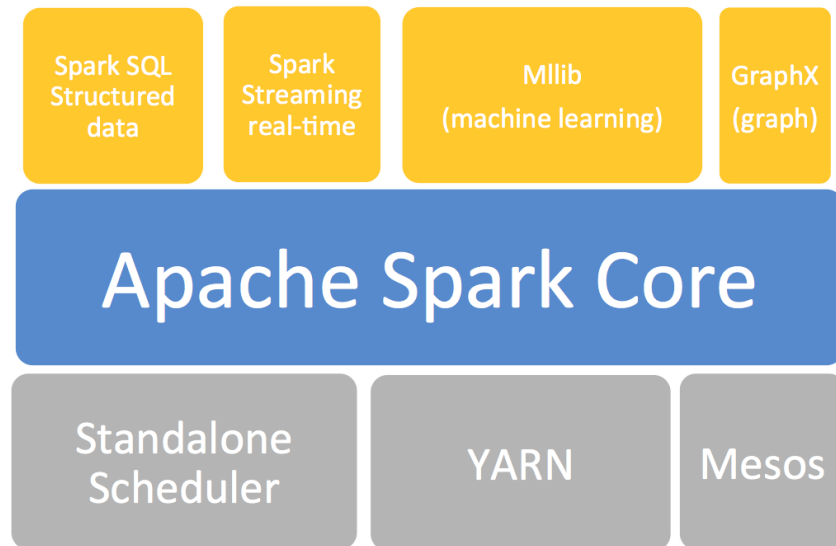


Fig. 2: The Spark stack

4. Runs Everywhere

Spark runs on Hadoop, Mesos, standalone, or in the cloud. It can access diverse data sources including HDFS, Cassandra, HBase, and S3.

2.2 Why Spark with Python (PySpark)?

No matter you like it or not, Python has been one of the most popular programming languages.



Fig. 3: The Spark platform

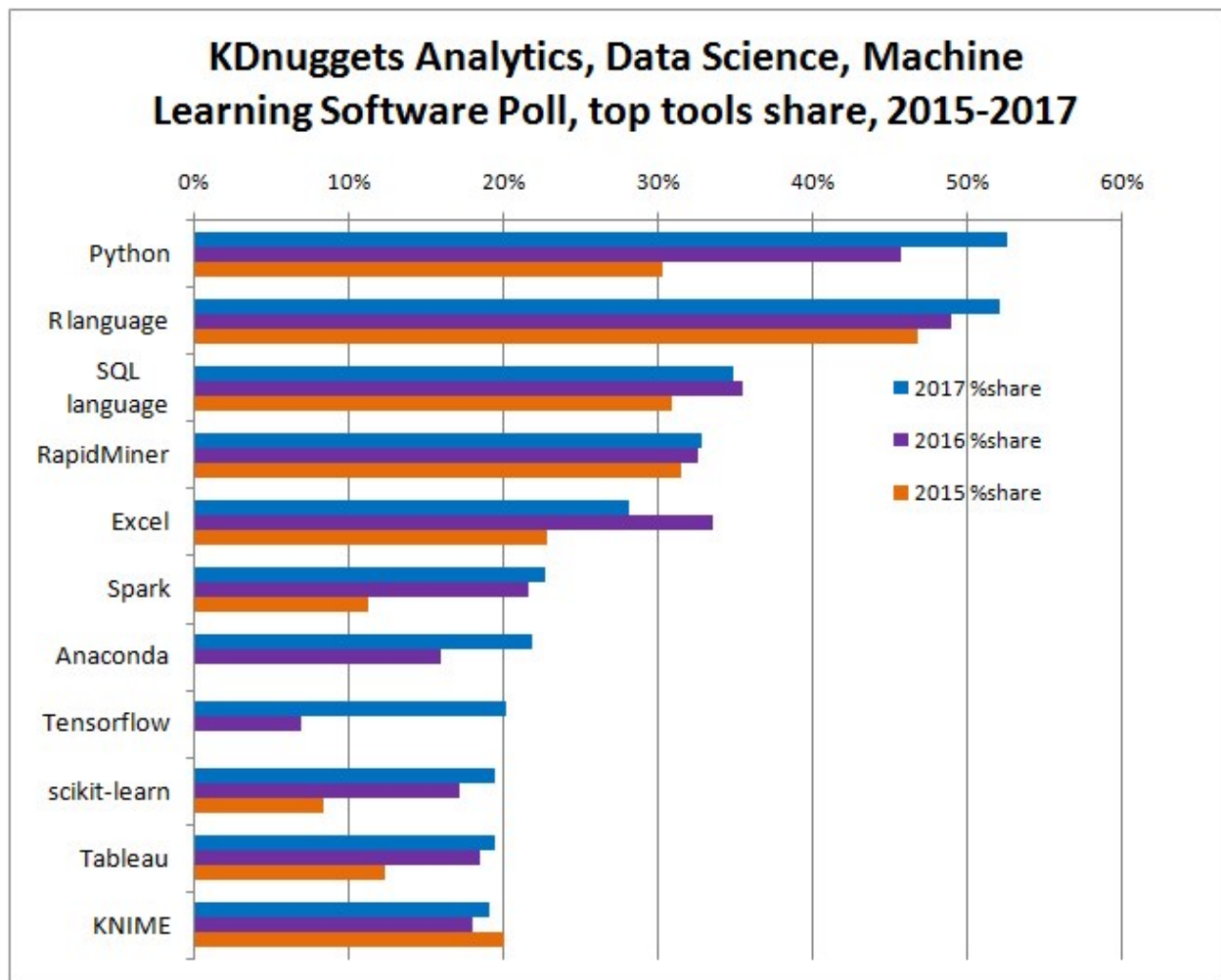


Fig. 4: KDnuggets Analytics/Data Science 2017 Software Poll from [kdnuggets](http://kdnuggets.com).

CONFIGURE RUNNING PLATFORM

Chinese proverb

Good tools are prerequisite to the successful execution of a job. – old Chinese proverb

A good programming platform can save you lots of troubles and time. Herein I will only present how to install my favorite programming platform and only show the easiest way which I know to set it up on Linux system. If you want to install on the other operator system, you can Google it. In this section, you may learn how to set up Pyspark on the corresponding programming platform and package.

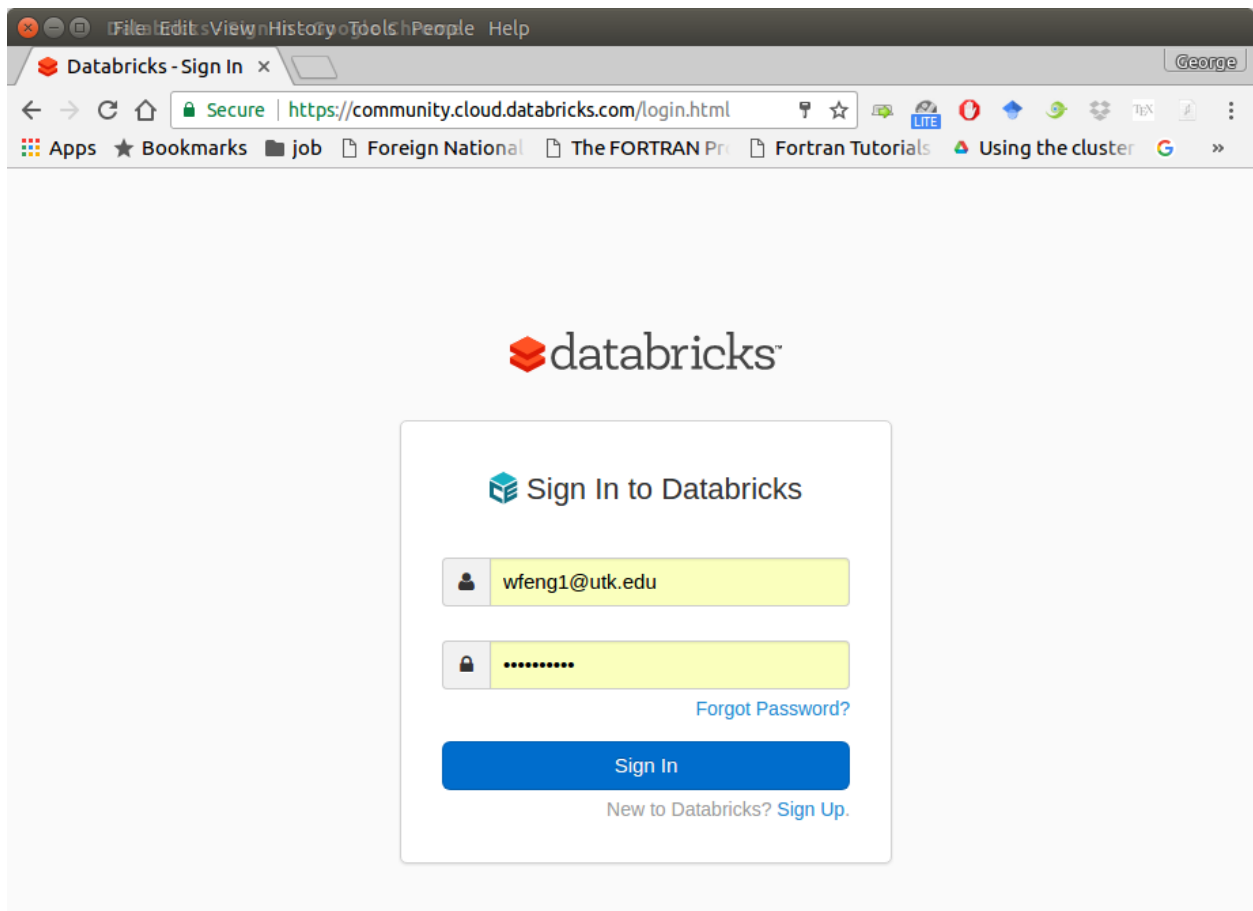
3.1 Run on Databricks Community Cloud

If you don't have any experience with Linux or Unix operator system, I would love to recommend you to use Spark on Databricks Community Cloud. Since you do not need to setup the Spark and it's totally **free** for Community Edition. Please follow the steps listed below.

1. Sign up a account at: <https://community.cloud.databricks.com/login.html>
2. Sign in with your account, then you can creat your cluster(machine), table(dataset) and notebook(code).
3. Create your cluster where your code will run
4. Import your dataset

Note: You need to save the path which appears at Uploaded to DBFS: /File-Store/tables/05rmhuqv1489687378010/. Since we will use this path to load the dataset.

5. Create your notebook



File Edit View History Tools People Help

Databricks

Secure | <https://community.cloud.databricks.com/?o=4622560542654492>

Apps Bookmarks job Foreign National The FORTRAN Pr Fortran Tutorials Using the cluster Attachments

Upgrade ? @George

Community Edition (2.45)

Welcome to databricks™

Featured Notebooks

- Introduction to Apache Spark on Databricks
- Databricks for Data Scientists
- Introduction to Structured Streaming

New

- Notebook
- Job
- Cluster
- Table
- Library

Documentation

- Databricks Guide
- Python, R, Scala, SQL
- Importing Data

Open Recent

- Databricks for Data Scientists
- linearRegression

What's new?

- Automatic termination of clusters
- Autoscaling local storage with EBS volumes (beta)

[Latest release notes](#)

Send Feedback

File Edit View History Tools People Help

Create Cluster - Databricks

Secure | <https://community.cloud.databricks.com/?o=4622560542654492#create/cluster>

Apps Bookmarks job Foreign National The FORTRAN Pr Fortran Tutorials Using the cluster Attachments

Upgrade ? @George

Create Cluster

New Cluster

Cancel Create Cluster

0 Workers: 0.0 GB Memory, 0 Cores, 0 DBU
1 Driver: 6.0 GB Memory, 0.88 Cores, 1 DBU

Cluster Name
MLmachine

Databricks Runtime Version
Spark 2.1 (Auto-updating, Scala 2.10)

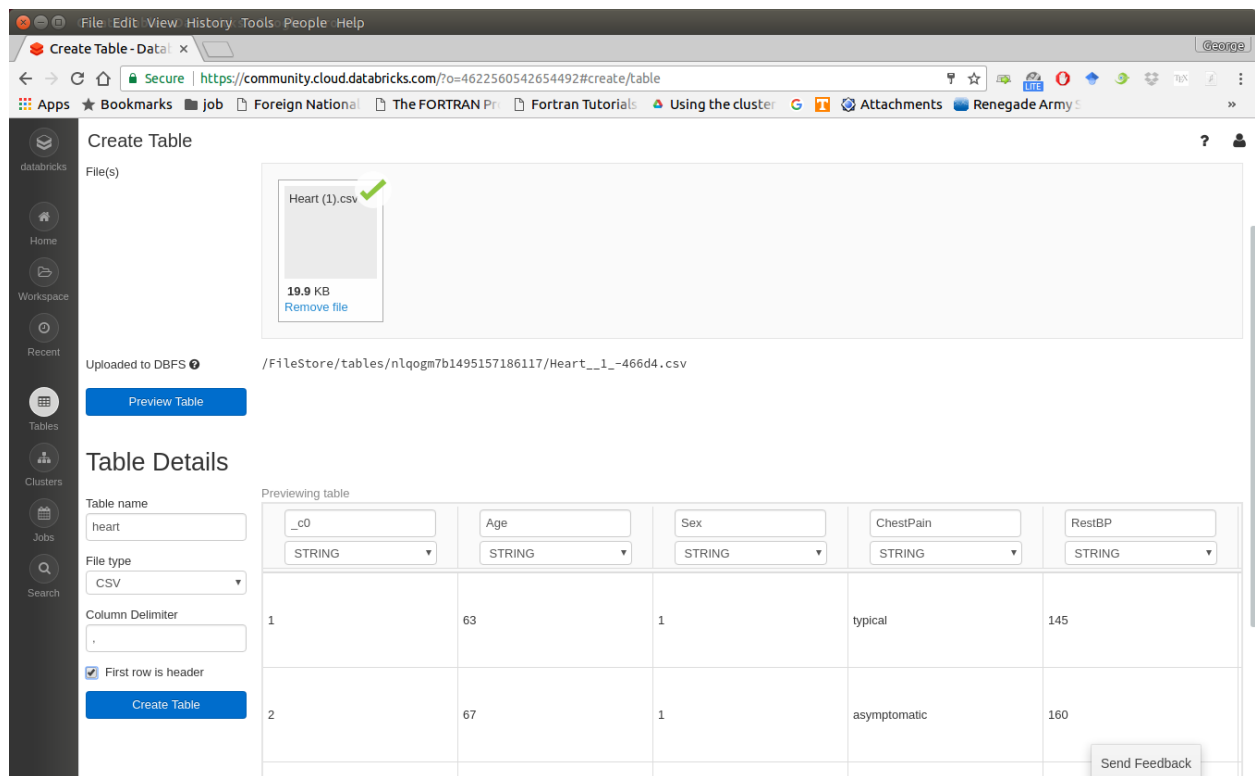
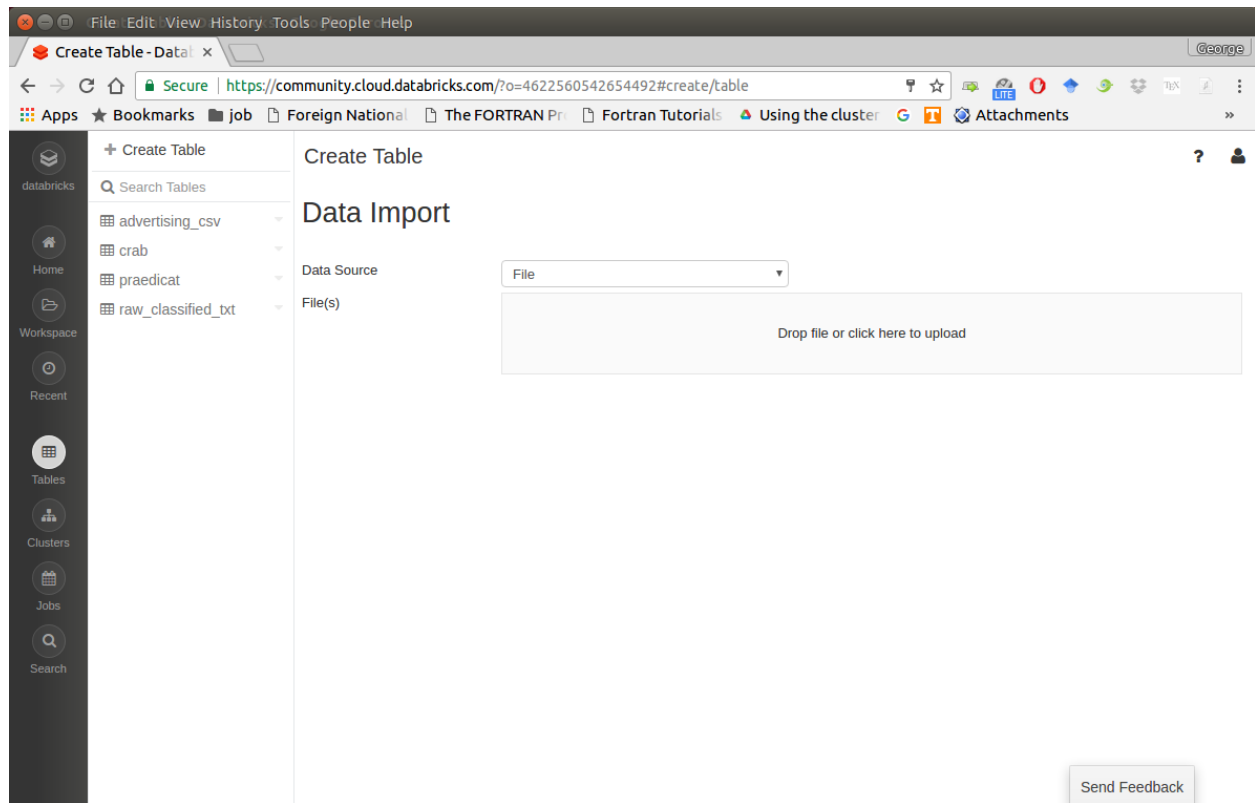
Instance

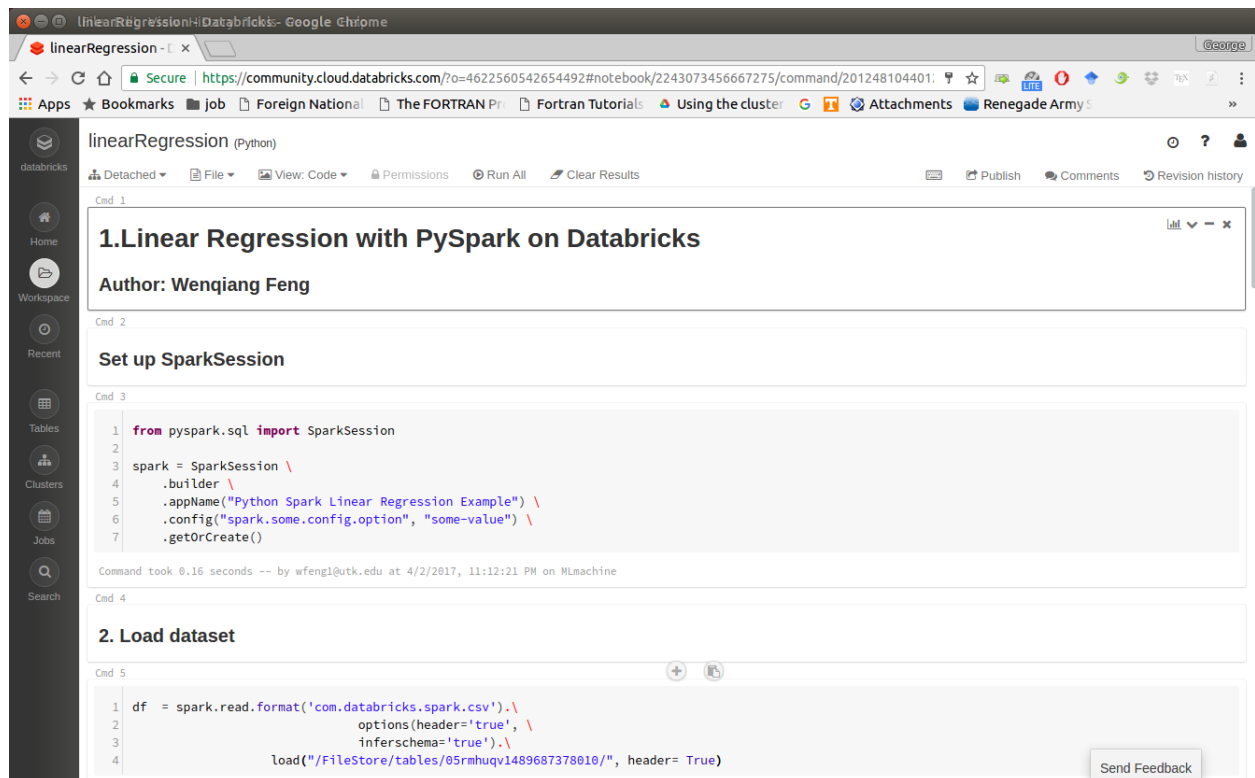
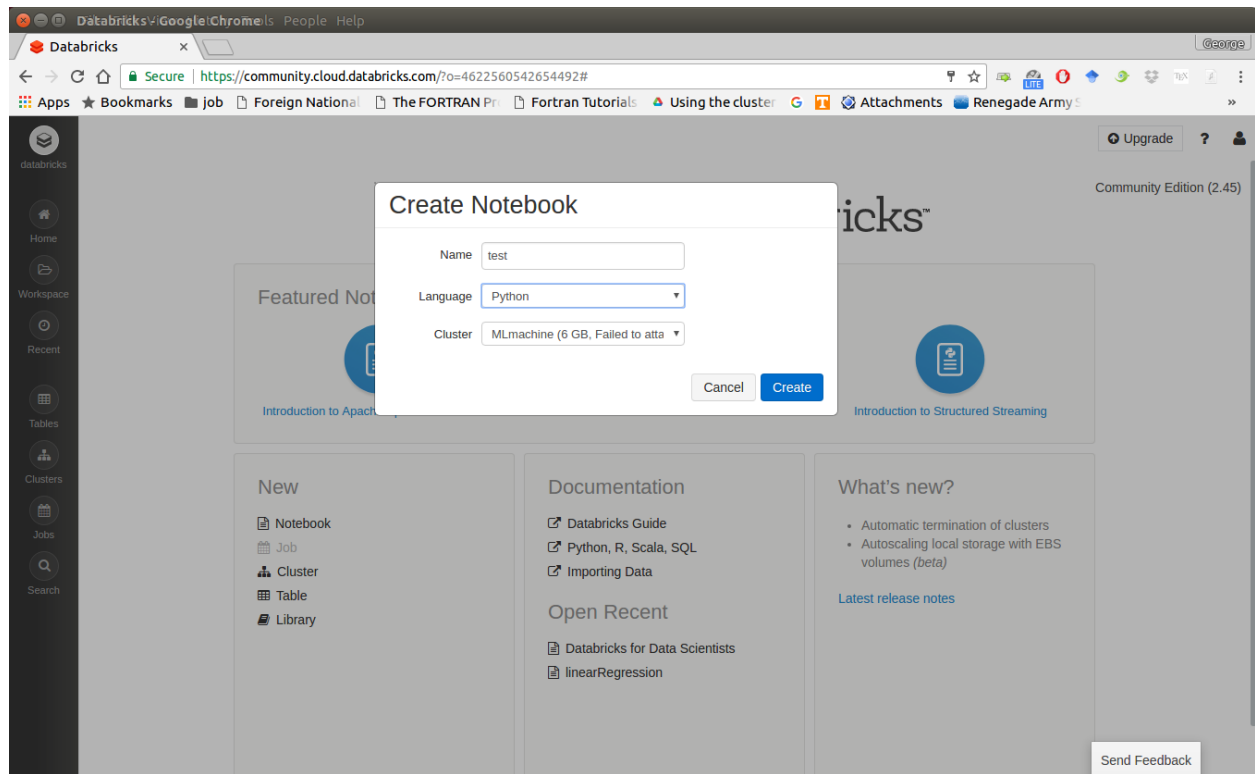
Free 6GB Memory: As a Community Edition user, your cluster will automatically terminate after an idle period of two hours. For [more configuration options](#), please [upgrade your Databricks subscription](#).

AWS Spark

Availability Zone
us-west-2c

Send Feedback





After finishing the above 5 steps, you are ready to run your Spark code on Databricks Community Cloud. I will run all the following demos on Databricks Community Cloud. Hopefully, when you run the demo code, you will get the following results:

```
+---+-----+-----+-----+-----+
|_c0|      TV|Radio|Newspaper|Sales|
+---+-----+-----+-----+-----+
|  1|230.1| 37.8|      69.2| 22.1|
|  2| 44.5| 39.3|      45.1| 10.4|
|  3| 17.2| 45.9|      69.3|  9.3|
|  4|151.5| 41.3|      58.5| 18.5|
|  5|180.8| 10.8|      58.4| 12.9|
+---+-----+-----+-----+-----+
only showing top 5 rows

root
|-- _c0: integer (nullable = true)
|-- TV: double (nullable = true)
|-- Radio: double (nullable = true)
|-- Newspaper: double (nullable = true)
|-- Sales: double (nullable = true)
```

3.2 Configure Spark on Mac and Ubuntu

3.2.1 Installing Prerequisites

I will strongly recommend you to install [Anaconda](#), since it contains most of the prerequisites and support multiple Operator Systems.

1. Install Python

Go to Ubuntu Software Center and follow the following steps:

- Open Ubuntu Software Center
- Search for python
- And click Install

Or Open your terminal and using the following command:

```
sudo apt-get install build-essential checkinstall
sudo apt-get install libreadline-gplv2-dev libncursesw5-dev libssl-dev
                        libsqlite3-dev tk-dev libgdbm-dev libc6-dev libbz2-dev
sudo apt-get install python
sudo easy_install pip
sudo pip install ipython
```

3.2.2 Install Java

Java is used by many other softwares. So it is quite possible that you have already installed it. You can by using the following command in Command Prompt:

```
java -version
```

Otherwise, you can follow the steps in [How do I install Java for my Mac?](#) to install java on Mac and use the following command in Command Prompt to install on Ubuntu:

```
sudo apt-add-repository ppa:webupd8team/java
sudo apt-get update
sudo apt-get install oracle-java8-installer
```

3.2.3 Install Java SE Runtime Environment

I installed ORACLE [Java JDK](#).

Warning: Installing Java and Java SE Runtime Environment steps are very important, since Spark is a domain-specific language written in Java.

You can check if your Java is available and find it's version by using the following command in Command Prompt:

```
java -version
```

If your Java is installed successfully, you will get the similar results as follows:

```
java version "1.8.0_131"
Java(TM) SE Runtime Environment (build 1.8.0_131-b11)
Java HotSpot(TM) 64-Bit Server VM (build 25.131-b11, mixed mode)
```

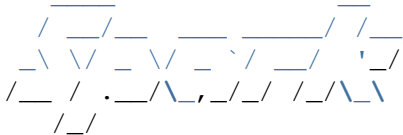
3.2.4 Install Apache Spark

Actually, the Pre-build version doesn't need installation. You can use it when you unpack it.

- Download: You can get the Pre-built Apache Spark™ from [Download Apache Spark™](#).
- Unpack: Unpack the Apache Spark™ to the path where you want to install the Spark.
- Test: Test the Prerequisites: change the direction `spark-#.#.#-bin-hadoop#.#/bin` and run

```
./pyspark
```

```
Python 2.7.13 |Anaconda 4.4.0 (x86_64)| (default, Dec 20 2016,  
↪23:05:08)  
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.57)] on darwin  
Type "help", "copyright", "credits" or "license" for more_  
↪information.  
Anaconda is brought to you by Continuum Analytics.  
Please check out: http://continuum.io/thanks and https://anaconda.org  
Using Spark's default log4j profile: org/apache/spark/log4j-defaults._  
↪properties  
Setting default log level to "WARN".  
To adjust logging level use sc.setLogLevel(newLevel). For SparkR,  
use setLogLevel(newLevel).  
17/08/30 13:30:12 WARN NativeCodeLoader: Unable to load native-hadoop  
library for your platform... using builtin-java classes where_  
↪applicable  
17/08/30 13:30:17 WARN ObjectStore: Failed to get database global_  
↪temp,  
returning NoSuchObjectException  
Welcome to
```



```
version 2.1.1
```

```
Using Python version 2.7.13 (default, Dec 20 2016 23:05:08)  
SparkSession available as 'spark'.
```

3.2.5 Configure the Spark

- a. **Mac Operator System:** open your `bash_profile` in Terminal

```
vim ~/.bash_profile
```

And add the following lines to your `bash_profile` (remember to change the path)

```
# add for spark
export SPARK_HOME=your_spark_installation_path
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PATH=$PATH:$SPARK_HOME/bin
export PYSARK_DRIVER_PYTHON="jupyter"
export PYSARK_DRIVER_PYTHON_OPTS="notebook"
```

At last, remember to source your `bash_profile`

```
source ~/.bash_profile
```

- b. Ubuntu Operator Sysytem:** open your `bashrc` in Terminal

```
vim ~/.bashrc
```


And add the following lines to your `bashrc` (remember to change the path)

```
# add for spark
export SPARK_HOME=your_spark_installation_path
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
export PATH=$PATH:$SPARK_HOME/bin
export PYSARK_DRIVE_PYTHON="jupyter"
export PYSARK_DRIVE_PYTHON_OPTS="notebook"
```

At last, remember to source your `bashrc`

```
source ~/.bashrc
```

3.3 Configure Spark on Windows

Installing open source software on Windows is always a nightmare for me. Thanks for Deelesh Mandloi. You can follow the detailed procedures in the blog [Getting Started with PySpark on Windows](#) to install the Apache Spark™ on your Windows Operator System.

3.4 PySpark With Text Editor or IDE

3.4.1 PySpark With Jupyter Notebook

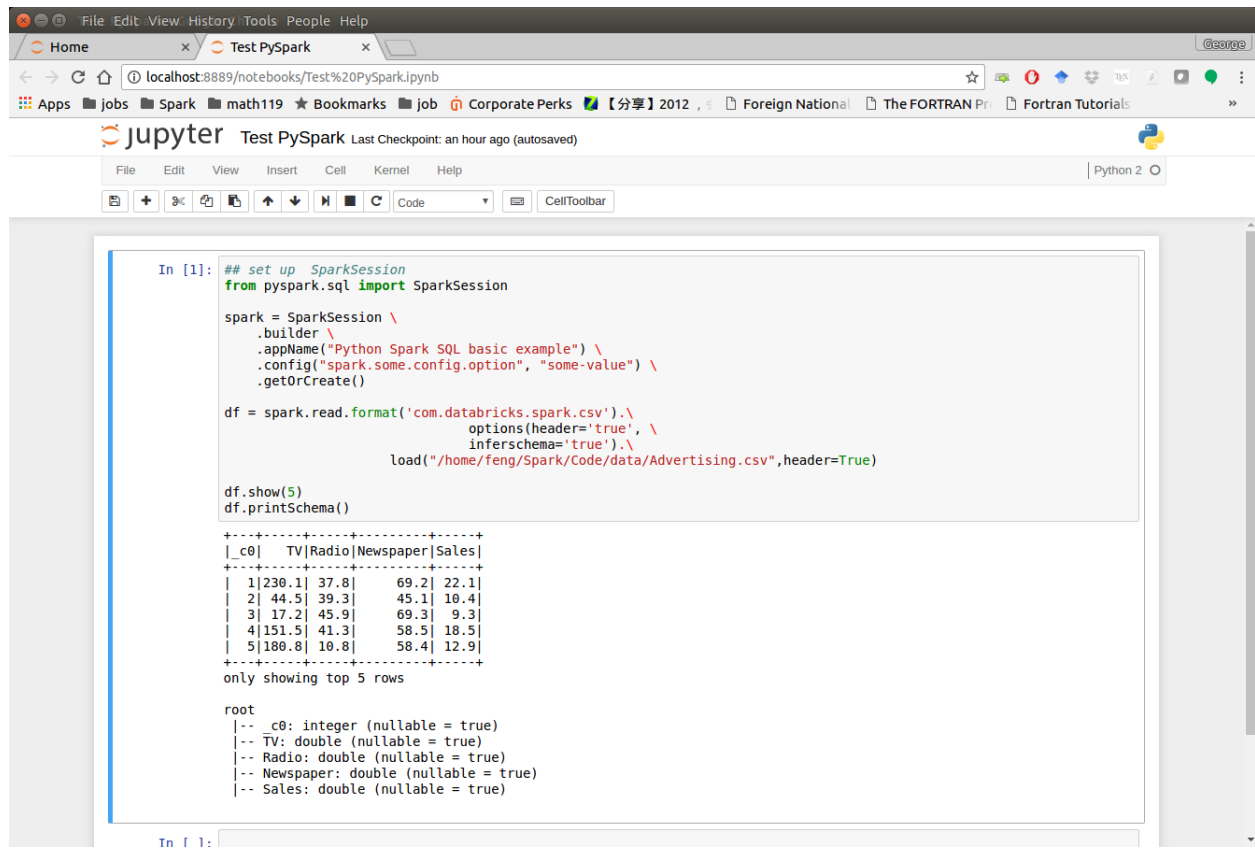
After you finishing the above setup steps in *Configure Spark on Mac and Ubuntu*, then you should be good to write and run your PySpark Code in Jupyter notebook.

3.4.2 PySpark With PyCharm

After you finishing the above setup steps in *Configure Spark on Mac and Ubuntu*, then you should be good to add the PySpark to your PyCharm project.

1. Create a new PyCharm project
2. Go to Project Structure
 - Option 1: File -> Settings -> Project: -> Project Structure
 - Option 2: PyCharm -> Preferences -> Project: -> Project Structure
3. Add Content Root: all ZIP files from `$SPARK_HOME/python/lib`
4. Run your script

Learning Apache Spark with Python



The image shows a Jupyter Notebook titled "Test PySpark" running in a web browser. The notebook contains a code cell with the following Python code:

```
In [1]: ## set up SparkSession
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark SQL basic example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

df = spark.read.format('com.databricks.spark.csv') \
    .options(header='true', \
              inferSchema='true') \
    .load("/home/feng/Spark/Code/data/Advertising.csv", header=True)

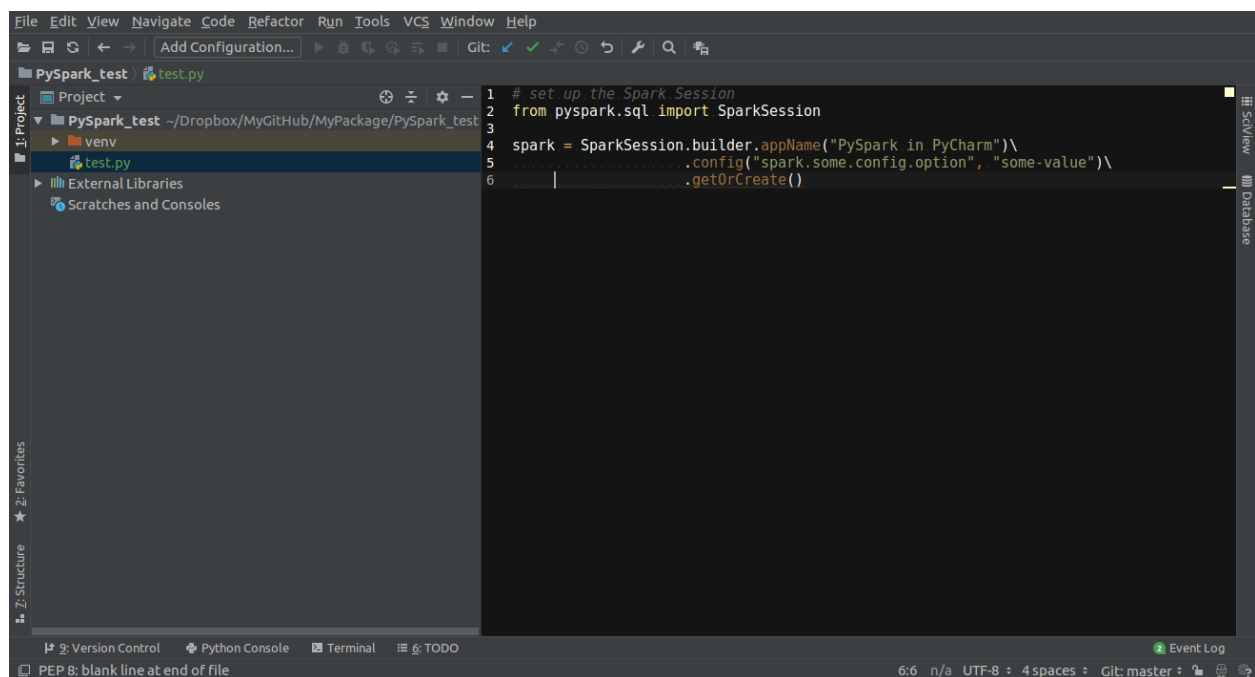
df.show(5)
df.printSchema()
```

The output of the code is a table showing the first 5 rows of the 'Advertising' dataset. The table has 5 columns: '_c0', 'TV', 'Radio', 'Newspaper', and 'Sales'. The data is as follows:

_c0	TV	Radio	Newspaper	Sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

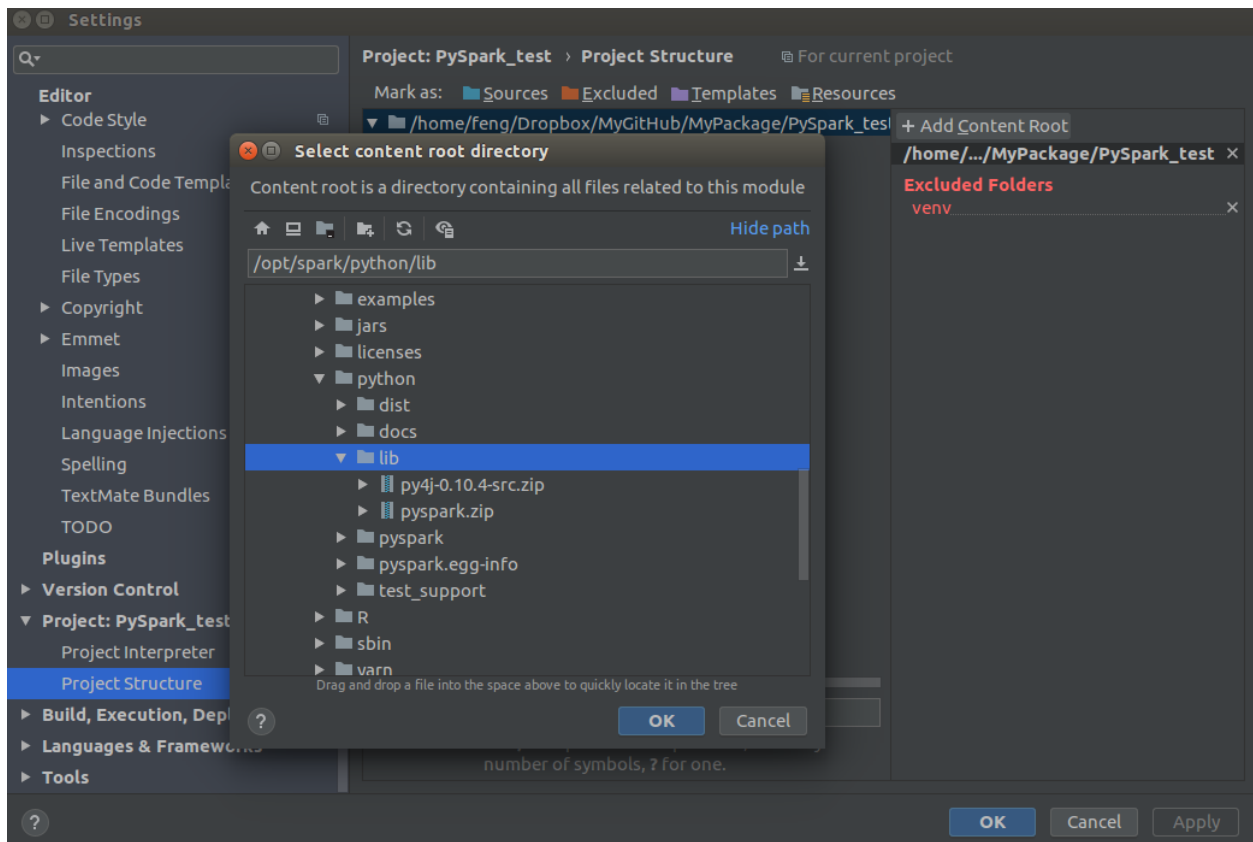
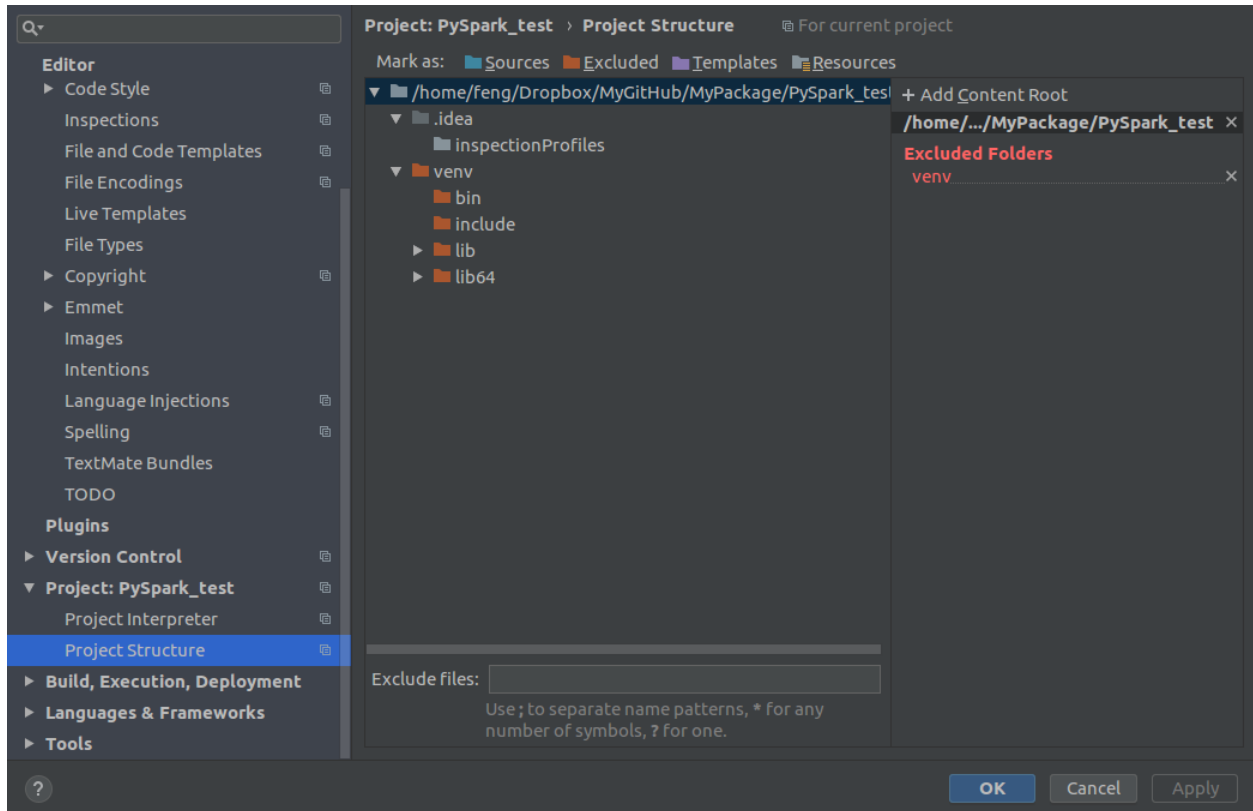
Below the table, the output indicates that only the top 5 rows are shown and provides the schema for the DataFrame:

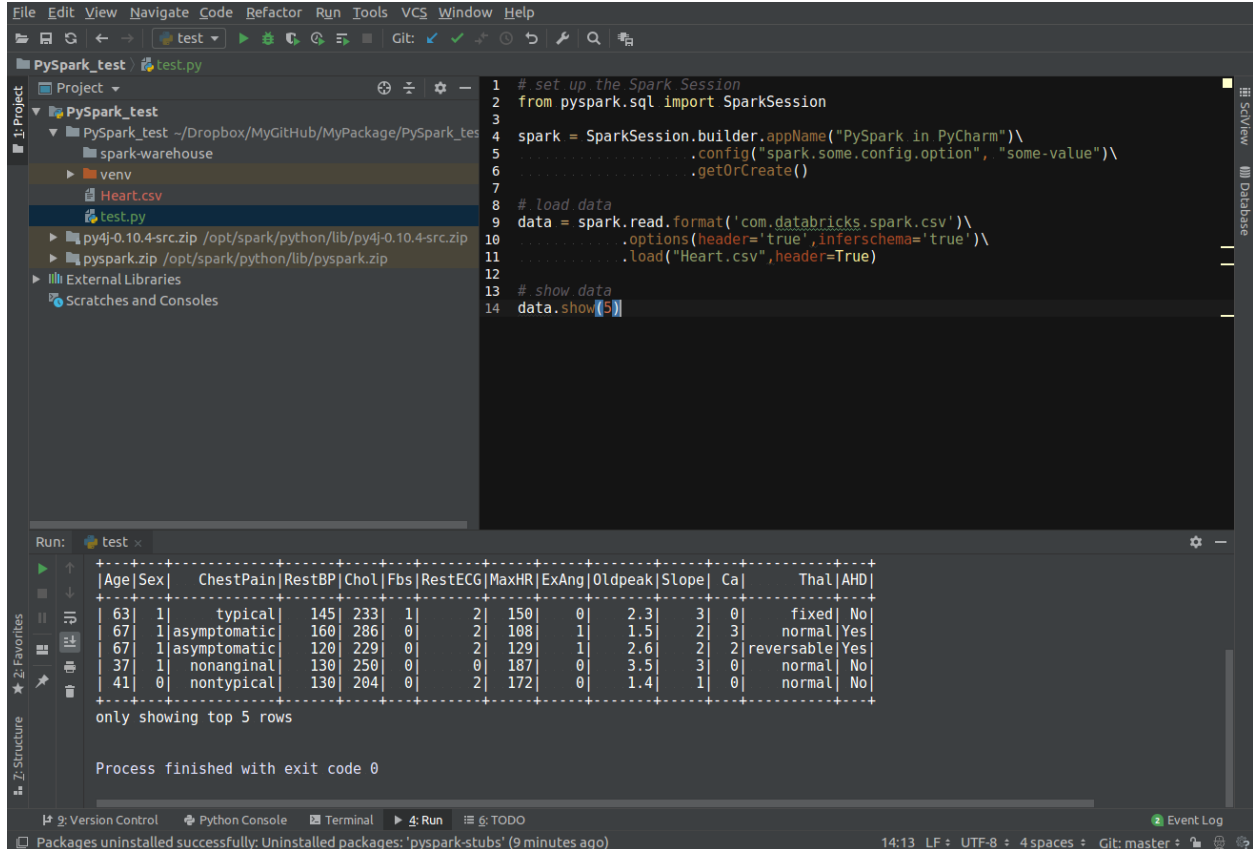
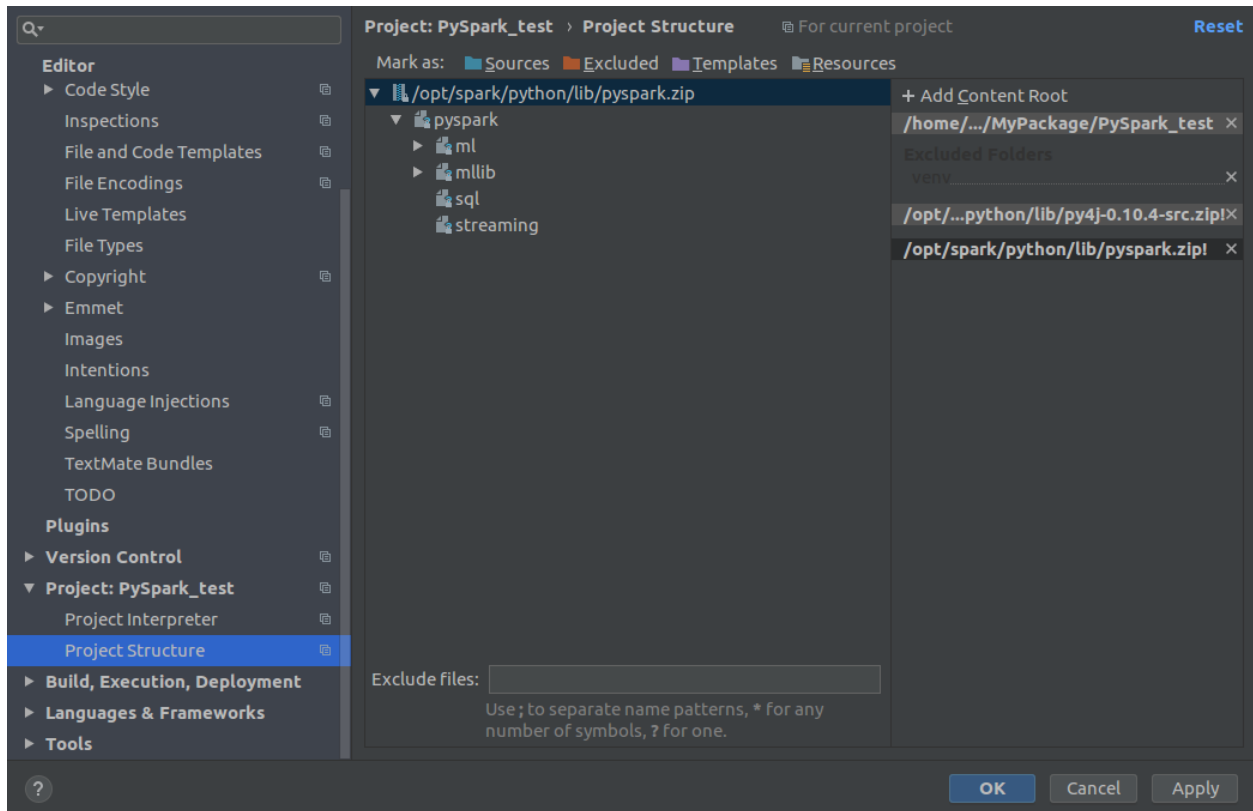
```
root
-- _c0: integer (nullable = true)
-- TV: double (nullable = true)
-- Radio: double (nullable = true)
-- Newspaper: double (nullable = true)
-- Sales: double (nullable = true)
```



The image shows the PyCharm IDE interface with a file named 'test.py' open. The code in the file is identical to the code in the Jupyter Notebook:

```
1 # set up the Spark Session
2 from pyspark.sql import SparkSession
3
4 spark = SparkSession.builder.appName("PySpark in PyCharm")\
5     .config("spark.some.config.option", "some-value")\
6     .getOrCreate()
```





3.4.3 PySpark With Apache Zeppelin

After you finishing the above setup steps in *Configure Spark on Mac and Ubuntu*, then you should be good to write and run your PySpark Code in Apache Zeppelin.

The screenshot shows the Apache Zeppelin Notebook interface. The top bar includes the Zeppelin logo, 'Notebook' and 'Job' tabs, a search bar, and a user profile. The main area displays three code executors, each with a 'test' label and a 'Head' dropdown.

Executor 1: Executes a PySpark script to load a CSV file from a local path and displays the first 4 rows of the resulting DataFrame.

```
df = spark.read.format('com.databricks.spark.csv').\
    options(header='true', \
    inferSchema='true').\
    load("/home/feng/Dropbox/MyTutorial/LearningApacheSpark/doc/data/bank.csv", header=True);
```

Output (showing top 4 rows):

age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
30	unemployed	married	primary	no	1787	no	no	cellular	19	oct	79	1	-1	0	unknown	no
33	services	married	secondary	no	4789	yes	yes	cellular	11	may	220	1	339	4	failure	no
35	management	single	tertiary	no	1350	yes	no	cellular	16	apr	185	1	330	1	failure	no
30	management	married	tertiary	no	1476	yes	yes	unknown	3	jun	199	4	-1	0	unknown	no

Executor 2: Registers the DataFrame as a temporary table named 'bank'.

```
df.registerTempTable("bank")
```

Executor 3: Contains three SQL queries, each with a corresponding visualization.

Query 1: Selects age and count(1) value from bank where age < 30, grouped by age and ordered by age. The visualization is a pie chart showing the distribution of ages.

Query 2: Selects age and count(1) value from bank where age < 30, grouped by age and ordered by age. The visualization is a pie chart showing the distribution of ages.

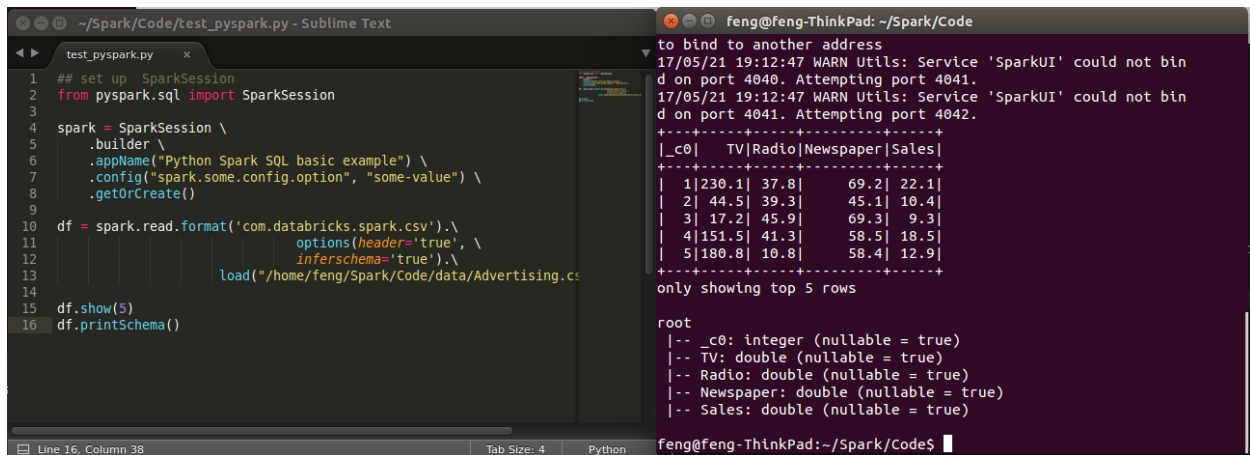
Query 3: Selects age and count(1) value from bank where marital = 'single', grouped by age. The visualization is a histogram showing the distribution of ages for single individuals.

3.4.4 PySpark With Sublime Text

After you finishing the above setup steps in *Configure Spark on Mac and Ubuntu*, then you should be good to use Sublime Text to write your PySpark Code and run your code as a normal python code in Terminal.

```
python test_pyspark.py
```

Then you should get the output results in your terminal.



```
test_pyspark.py
1  ## set up SparkSession
2  from pyspark.sql import SparkSession
3
4  spark = SparkSession \
5      .builder \
6      .appName("Python Spark SQL basic example") \
7      .config("spark.some.config.option", "some-value") \
8      .getOrCreate()
9
10 df = spark.read.format('com.databricks.spark.csv').\
11     options(header='true', \
12             inferSchema='true').\
13     load("/home/feng/Spark/Code/data/Advertising.csv", header=True)
14
15 df.show(5)
16 df.printSchema()
```

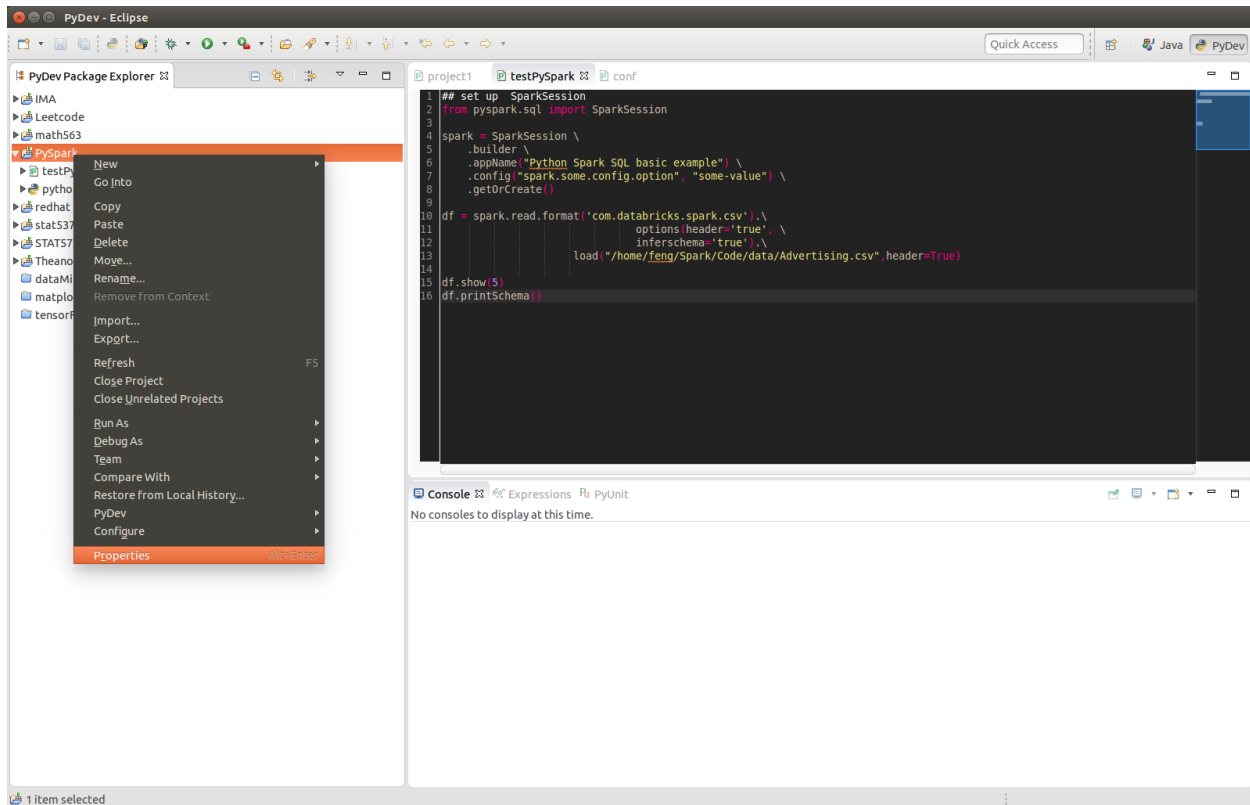
```
feng@feng-ThinkPad: ~/Spark/Code
to bind to another address
17/05/21 19:12:47 WARN Utils: Service 'SparkUI' could not bind
on port 4040. Attempting port 4041.
17/05/21 19:12:47 WARN Utils: Service 'SparkUI' could not bind
on port 4041. Attempting port 4042.
-----+-----+-----+-----+
|_c0|  TV|Radio|Newspaper|Sales|
-----+-----+-----+-----+
| 1|230.1| 37.8|   69.2|  22.1|
| 2| 44.5| 39.3|   45.1|  10.4|
| 3| 17.2| 45.9|   69.3|   9.3|
| 4|151.5| 41.3|   58.5|  18.5|
| 5|180.8| 10.8|   58.4|  12.9|
-----+-----+-----+-----+
only showing top 5 rows

root
|-- _c0: integer (nullable = true)
|-- TV: double (nullable = true)
|-- Radio: double (nullable = true)
|-- Newspaper: double (nullable = true)
|-- Sales: double (nullable = true)
```

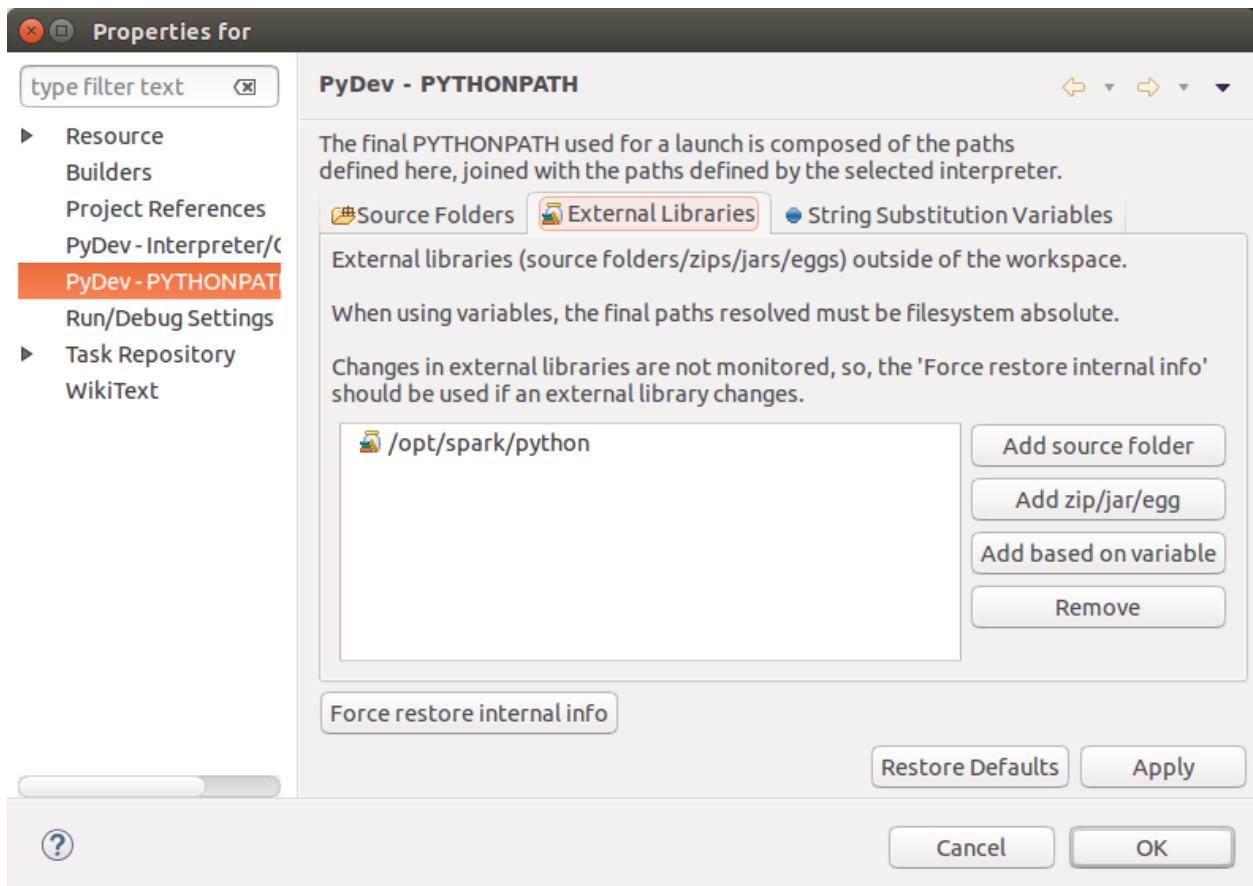
3.4.5 PySpark With Eclipse

If you want to run PySpark code on Eclipse, you need to add the paths for the **External Libraries** for your **Current Project** as follows:

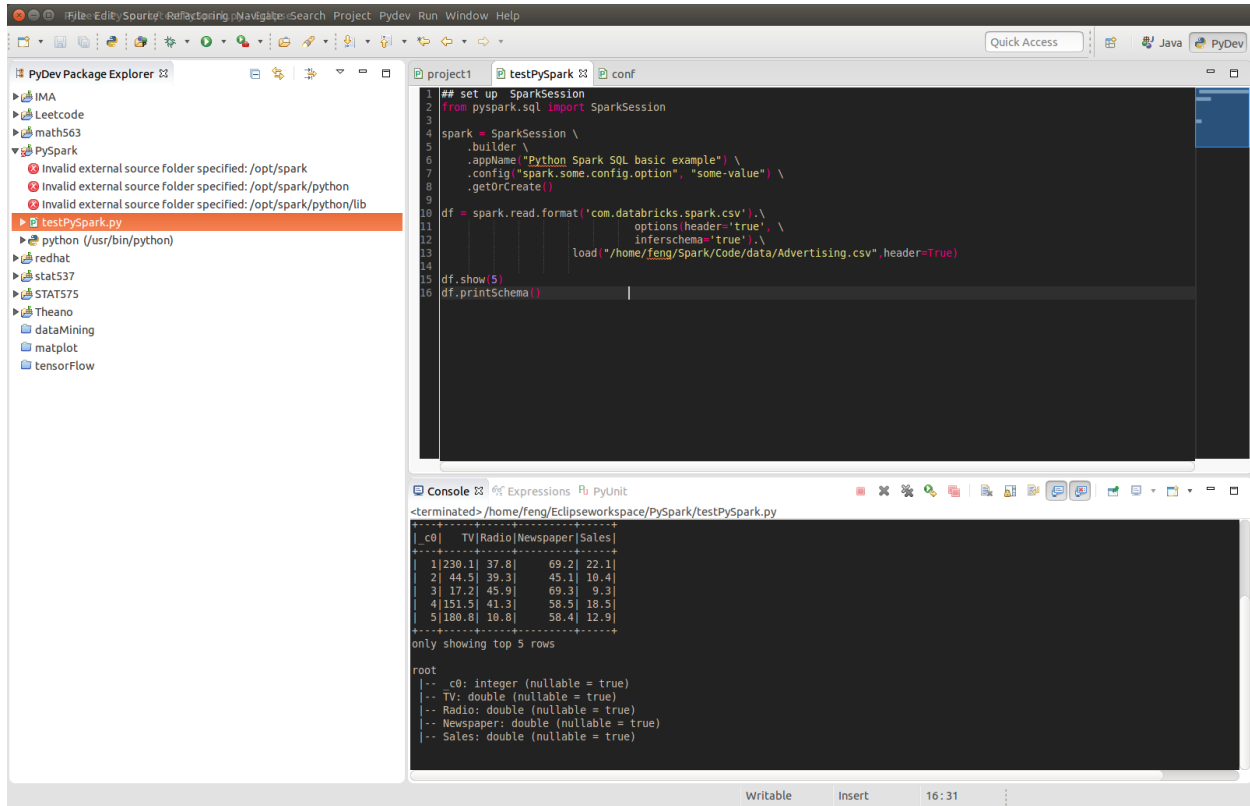
1. Open the properties of your project



2. Add the paths for the **External Libraries**



And then you should be good to run your code on Eclipse with PyDev.



3.5 PySparkling Water: Spark + H2O

1. Download Sparkling Water from: <https://s3.amazonaws.com/h2o-release/sparkling-water/rel-2.4/5/index.html>
2. Test PySparkling

```
unzip sparkling-water-2.4.5.zip
cd ~/sparkling-water-2.4.5/bin
./pysparkling
```

If you have a correct setup for PySpark, then you will get the following results:

```
Using Spark defined in the SPARK_HOME=/Users/dt216661/spark environmental_
↪property

Python 3.7.1 (default, Dec 14 2018, 13:28:58)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.57)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
2019-02-15 14:08:30 WARN NativeCodeLoader:62 - Unable to load native-hadoop_
↪library for your platform... using builtin-java classes where applicable
```

(continues on next page)

3.7 PySpark on Colaboratory

Colaboratory is a free Jupyter notebook environment that requires no setup and runs entirely in the cloud.

3.7.1 Installation

```
!pip install pyspark
```

3.7.2 Testing

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

df = spark.sparkContext\
    .parallelize([(1, 2, 3, 'a b c'),
                  (4, 5, 6, 'd e f'),
                  (7, 8, 9, 'g h i')])\
    .toDF(['col1', 'col2', 'col3', 'col4'])

df.show()
```

Output:

```
+----+----+----+----+
|col1|col2|col3| col4|
+----+----+----+----+
|   1|   2|   3|a b c|
|   4|   5|   6|d e f|
|   7|   8|   9|g h i|
+----+----+----+----+
```

3.8 Demo Code in this Section

The Jupyter notebook can be download from [installation on colab](#).

- Python Source code

```
## set up SparkSession
from pyspark.sql import SparkSession

spark = SparkSession \
```

(continues on next page)

(continued from previous page)

```
.builder \
.appName("Python Spark SQL basic example") \
.config("spark.some.config.option", "some-value") \
.getOrCreate()

df = spark.read.format('com.databricks.spark.csv').\
    options(header='true', \
             inferSchema='true').\
    load("/home/feng/Spark/Code/data/Advertising.csv", header=True)

df.show(5)
df.printSchema()
```


AN INTRODUCTION TO APACHE SPARK

Chinese proverb

Know yourself and know your enemy, and you will never be defeated – idiom, from Sunzi's Art of War

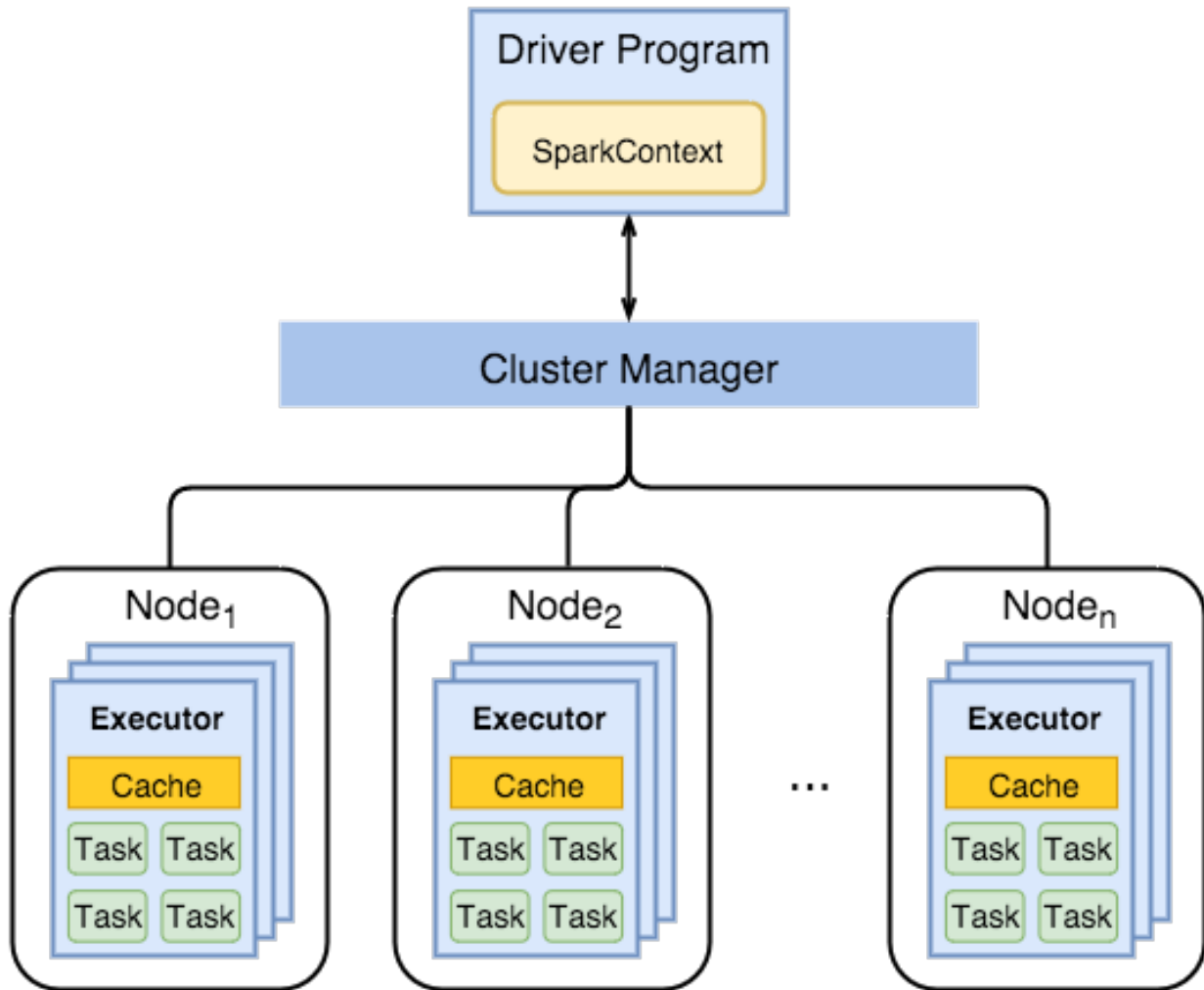
4.1 Core Concepts

Most of the following content comes from [\[Kirillov2016\]](#). So the copyright belongs to **Anton Kirillov**. I will refer you to get more details from [Apache Spark core concepts, architecture and internals](#).

Before diving deep into how Apache Spark works, let's understand the jargon of Apache Spark

- **Job:** A piece of code which reads some input from HDFS or local, performs some computation on the data and writes some output data.
- **Stages:** Jobs are divided into stages. Stages are classified as a Map or reduce stages (It's easier to understand if you have worked on Hadoop and want to correlate). Stages are divided based on computational boundaries, all computations (operators) cannot be updated in a single Stage. It happens over many stages.
- **Tasks:** Each stage has some tasks, one task per partition. One task is executed on one partition of data on one executor (machine).
- **DAG:** DAG stands for Directed Acyclic Graph, in the present context it's a DAG of operators.
- **Executor:** The process responsible for executing a task.
- **Master:** The machine on which the Driver program runs
- **Slave:** The machine on which the Executor program runs

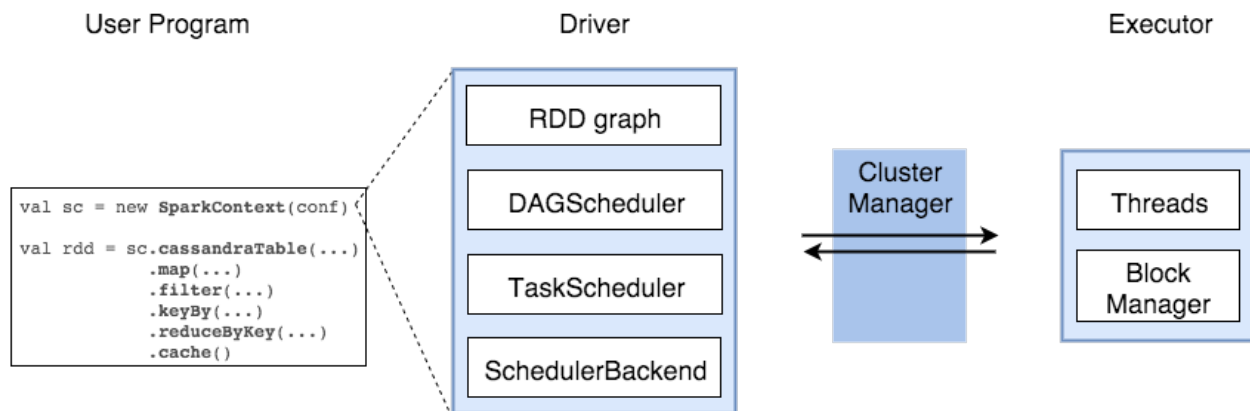
4.2 Spark Components



1. Spark Driver
 - separate process to execute user applications
 - creates SparkContext to schedule jobs execution and negotiate with cluster manager
2. Executors
 - run tasks scheduled by driver
 - store computation results in memory, on disk or off-heap
 - interact with storage systems
3. Cluster Manager
 - Mesos
 - YARN

- Spark Standalone

Spark Driver contains more components responsible for translation of user code into actual jobs executed on cluster:



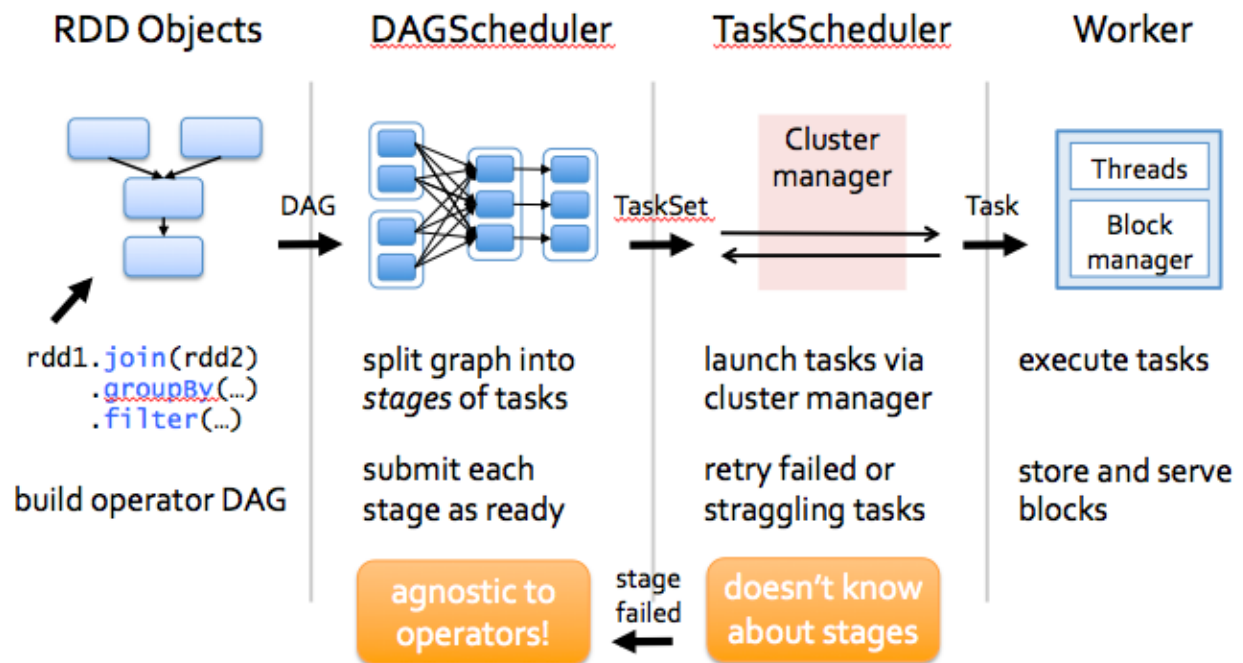
- **SparkContext**
 - represents the connection to a Spark cluster, and can be used to create RDDs, accumulators and broadcast variables on that cluster
- **DAGScheduler**
 - computes a DAG of stages for each job and submits them to TaskScheduler determines preferred locations for tasks (based on cache status or shuffle files locations) and finds minimum schedule to run the jobs
- **TaskScheduler**
 - responsible for sending tasks to the cluster, running them, retrying if there are failures, and mitigating stragglers
- **SchedulerBackend**
 - backend interface for scheduling systems that allows plugging in different implementations (Mesos, YARN, Standalone, local)
- **BlockManager**
 - provides interfaces for putting and retrieving blocks both locally and remotely into various stores (memory, disk, and off-heap)

4.3 Architecture

4.4 How Spark Works?

Spark has a small code base and the system is divided in various layers. Each layer has some responsibilities. The layers are independent of each other.

The first layer is the interpreter, Spark uses a Scala interpreter, with some modifications. As you enter your code in spark console (creating RDD's and applying operators), Spark creates an operator graph. When the user runs an action (like collect), the Graph is submitted to a DAG Scheduler. The DAG scheduler divides operator graph into (map and reduce) stages. A stage is comprised of tasks based on partitions of the input data. The DAG scheduler pipelines operators together to optimize the graph. For e.g. Many map operators can be scheduled in a single stage. This optimization is key to Spark's performance. The final result of a DAG scheduler is a set of stages. The stages are passed on to the Task Scheduler. The task scheduler launches tasks via cluster manager. (Spark Standalone/Yarn/Mesos). The task scheduler doesn't know about dependencies among stages.



PROGRAMMING WITH RDDS

Chinese proverb

If you only know yourself, but not your opponent, you may win or may lose. If you know neither yourself nor your enemy, you will always endanger yourself – idiom, from Sunzi's Art of War

RDD represents **Resilient Distributed Dataset**. An RDD in Spark is simply an immutable distributed collection of objects sets. Each RDD is split into multiple partitions (similar pattern with smaller sets), which may be computed on different nodes of the cluster.

5.1 Create RDD

Usually, there are two popular ways to create the RDDs: loading an external dataset, or distributing a set of collection of objects. The following examples show some simplest ways to create RDDs by using `parallelize()` function which takes an already existing collection in your program and pass the same to the Spark Context.

1. By using `parallelize()` function

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

df = spark.sparkContext.parallelize([(1, 2, 3, 'a b c'),
                                     (4, 5, 6, 'd e f'),
                                     (7, 8, 9, 'g h i')]).toDF(['col1', 'col2', 'col3', 'col4'])
```

Then you will get the RDD data:

```
df.show()

+----+----+----+----+
```

(continues on next page)

(continued from previous page)

```
|col1|col2|col3| col4|
+----+----+----+-----+
|  1 |  2 |  3 |a b c|
|  4 |  5 |  6 |d e f|
|  7 |  8 |  9 |g h i|
+----+----+----+-----+
```

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

myData = spark.sparkContext.parallelize([(1,2), (3,4), (5,6), (7,8), (9,10)])
```

Then you will get the RDD data:

```
myData.collect()

[(1, 2), (3, 4), (5, 6), (7, 8), (9, 10)]
```

2. By using createDataFrame() function

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

Employee = spark.createDataFrame([
    ('1', 'Joe', '70000', '1'),
    ('2', 'Henry', '80000', '2'),
    ('3', 'Sam', '60000', '2'),
    ('4', 'Max', '90000', '1')],
    ['Id', 'Name', 'Sallary', 'DepartmentId']
)
```

Then you will get the RDD data:

```
+---+-----+-----+-----+
| Id| Name|Sallary|DepartmentId|
+---+-----+-----+-----+
|  1|  Joe| 70000|           1|
|  2|Henry| 80000|           2|
|  3|  Sam| 60000|           2|
|  4|  Max| 90000|           1|
+---+-----+-----+-----+
```

3. By using read and load functions

a. Read dataset from .csv file

```
## set up SparkSession
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
    .getOrCreate()

df = spark.read.format('com.databricks.spark.csv').\
    options(header='true', \
             inferSchema='true').\
    load("/home/feng/Spark/Code/data/Advertising.csv",
        header=True)

df.show(5)
df.printSchema()
```

Then you will get the RDD data:

```
+---+-----+-----+-----+-----+
|_c0|    TV|Radio|Newspaper|Sales|
+---+-----+-----+-----+-----+
|  1|230.1| 37.8|    69.2| 22.1|
|  2| 44.5| 39.3|    45.1| 10.4|
|  3| 17.2| 45.9|    69.3|  9.3|
|  4|151.5| 41.3|    58.5| 18.5|
|  5|180.8| 10.8|    58.4| 12.9|
+---+-----+-----+-----+-----+
only showing top 5 rows

root
 |-- _c0: integer (nullable = true)
 |-- TV: double (nullable = true)
 |-- Radio: double (nullable = true)
 |-- Newspaper: double (nullable = true)
 |-- Sales: double (nullable = true)
```

Once created, RDDs offer two types of operations: transformations and actions.

b. Read dataset from DataBase

```
## set up SparkSession
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName("Python Spark create RDD example") \
    .config("spark.some.config.option", "some-value") \
```

(continues on next page)

(continued from previous page)

```

        .getOrCreate()

## User information
user = 'your_username'
pw   = 'your_password'

## Database information
table_name = 'table_name'
url = 'jdbc:postgresql://##.##.##.##:5432/dataset?user='+user+'&
↳password='+pw
properties = {'driver': 'org.postgresql.Driver', 'password': pw, 'user'
↳': user}

df = spark.read.jdbc(url=url, table=table_name,
↳properties=properties)

df.show(5)
df.printSchema()

```

Then you will get the RDD data:

```

+---+-----+-----+-----+-----+
|_c0|    TV|Radio|Newspaper|Sales|
+---+-----+-----+-----+
|  1|230.1| 37.8|    69.2| 22.1|
|  2| 44.5| 39.3|    45.1| 10.4|
|  3| 17.2| 45.9|    69.3|  9.3|
|  4|151.5| 41.3|    58.5| 18.5|
|  5|180.8| 10.8|    58.4| 12.9|
+---+-----+-----+-----+
only showing top 5 rows

root
 |-- _c0: integer (nullable = true)
 |-- TV: double (nullable = true)
 |-- Radio: double (nullable = true)
 |-- Newspaper: double (nullable = true)
 |-- Sales: double (nullable = true)

```

Note: Reading tables from Database needs the proper drive for the corresponding Database. For example, the above demo needs `org.postgresql.Driver` and you need to download it and put it in `jars` folder of your spark installation path. I download `postgresql-42.1.1.jar` from the official website and put it in `jars` folder.

C. Read dataset from HDFS

```

from pyspark.conf import SparkConf
from pyspark.context import SparkContext
from pyspark.sql import HiveContext

```

(continues on next page)

(continued from previous page)

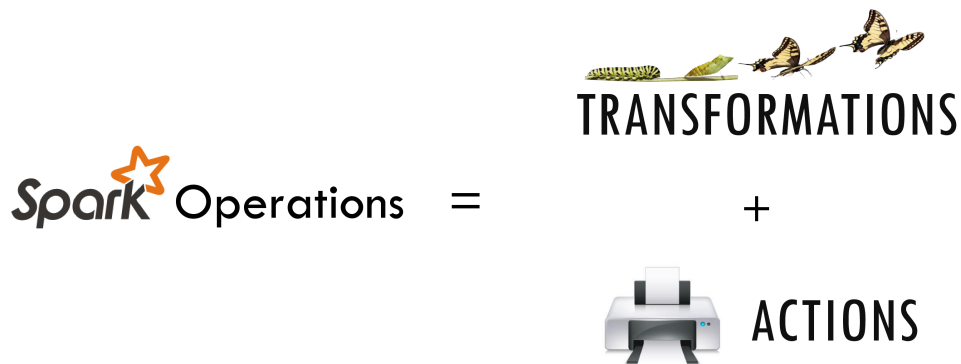
```
sc= SparkContext('local','example')
hc = HiveContext(sc)
tfl = sc.textFile("hdfs://cdhstltest/user/data/demo.CSV")
print(tfl.first())

hc.sql("use intg_cme_w")
spf = hc.sql("SELECT * FROM spf LIMIT 100")
print(spf.show(5))
```

5.2 Spark Operations

Warning: All the figures below are from Jeffrey Thompson. The interested reader is referred to [pyspark pictures](#)

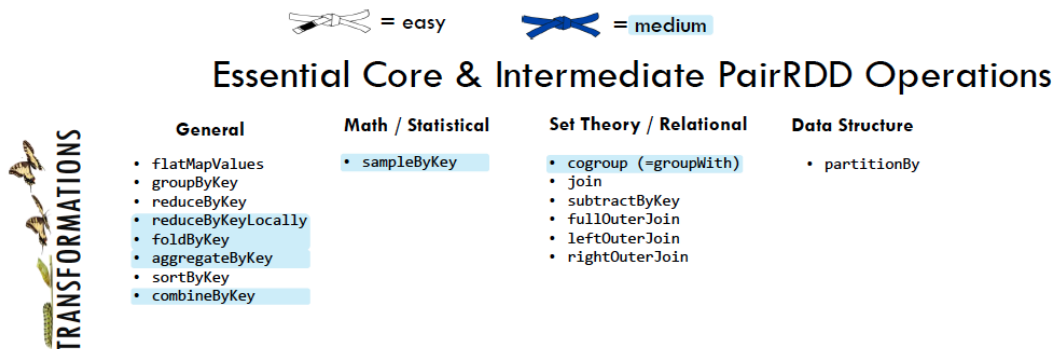
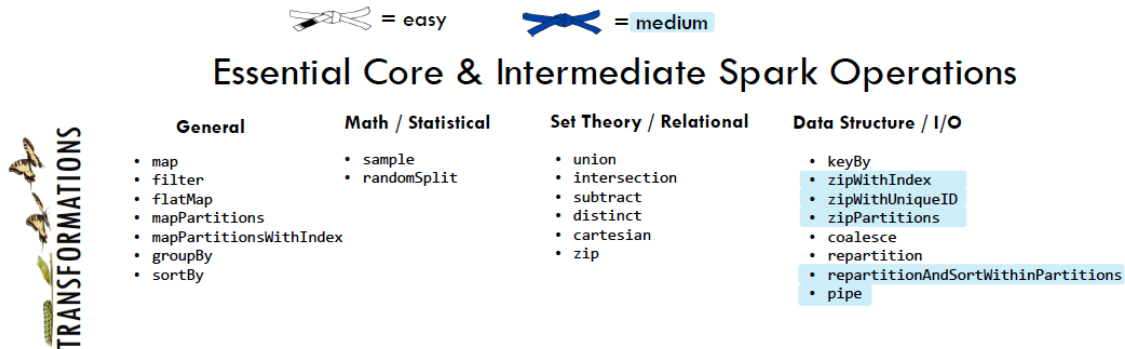
There are two main types of Spark operations: Transformations and Actions [[Karau2015](#)].



Note: Some people defined three types of operations: Transformations, Actions and Shuffles.

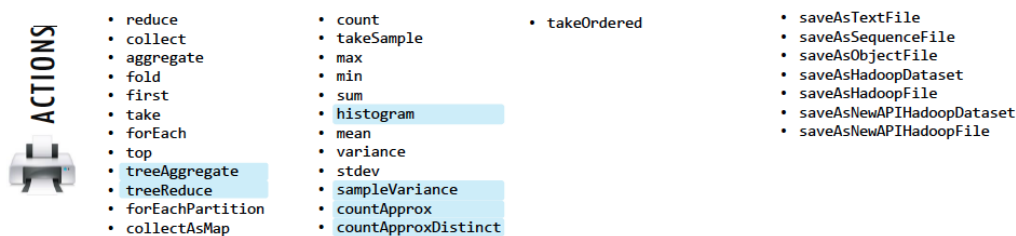
5.2.1 Spark Transformations

Transformations construct a new RDD from a previous one. For example, one common transformation is filtering data that matches a predicate.



5.2.2 Spark Actions

Actions, on the other hand, compute a result based on an RDD, and either return it to the driver program or save it to an external storage system (e.g., HDFS).





- keys
- values

- countByKey
- countByValue
- countByValueApprox
- countApproxDistinctByKey
- countApproxDistinctByValue
- countByKeyApprox
- sampleByKeyExact

5.3 rdd.DataFrame VS pd.DataFrame

5.3.1 Create DataFrame

1. From List

```
my_list = [['a', 1, 2], ['b', 2, 3], ['c', 3, 4]]
col_name = ['A', 'B', 'C']
```

:: Python Code:

```
# caution for the columns=
pd.DataFrame(my_list, columns= col_name)
#
spark.createDataFrame(my_list, col_name).show()
```

:: Comparison:

	A	B	C
0	a	1	2
1	b	2	3
2	c	3	4

Attention: Pay attention to the parameter `columns=` in `pd.DataFrame`. Since the default value will make the list as rows.

:: Python Code:

```
# caution for the columns=
pd.DataFrame(my_list, columns= col_name)
#
pd.DataFrame(my_list, col_name)
```

:: Comparison:

	A	B	C
0	a	1	2
1	b	2	3
2	c	3	4

2. From Dict

```
d = {'A': [0, 1, 0],
      'B': [1, 0, 1],
      'C': [1, 0, 0]}
```

:: Python Code:

```
pd.DataFrame(d) for
# Tedious for PySpark
spark.createDataFrame(np.array(list(d.values())).T.tolist(), list(d.keys())).
→ show()
```

:: Comparison:

	A	B	C
0	0	1	1
1	1	0	0
2	0	1	0

	A	B	C
0	0	1	1
1	1	0	0
2	0	1	0

5.3.2 Load DataFrame

1. From DataBase

Most of time, you need to share your code with your colleagues or release your code for Code Review or Quality assurance(QA). You will definitely do not want to have your User Information in the code. So you can save them in login.txt:

```
runawayhorse001
PythonTips
```

and use the following code to import your User Information:

```
#User Information
try:
    login = pd.read_csv(r'login.txt', header=None)
    user = login[0][0]
    pw = login[0][1]
    print('User information is ready!')
except:
    print('Login information is not available!!!')

#Database information
host = '##.##.##.##'
db_name = 'db_name'
table_name = 'table_name'
```


:: Comparison:

```
conn = psycopg2.connect(host=host, database=db_name, user=user, password=pw)
cur = conn.cursor()

sql = """
    select *
    from {table_name}
    """.format(table_name=table_name)
dp = pd.read_sql(sql, conn)
```

```
# connect to database
url = 'jdbc:postgresql://'+host+':5432/'+db_name+'?user='+user+'&password='+pw
properties = {'driver': 'org.postgresql.Driver', 'password': pw, 'user': user}
ds = spark.read.jdbc(url=url, table=table_name, properties=properties)
```

Attention: Reading tables from Database with PySpark needs the proper drive for the corresponding Database. For example, the above demo needs org.postgresql.Driver and you need to download it and put it in jars folder of your spark installation path. I download postgresql-42.1.1.jar from the official website and put it in jars folder.

2. From .csv

:: Comparison:

```
# pd.DataFrame dp: DataFrame pandas
dp = pd.read_csv('Advertising.csv')
# rdd.DataFrame. dp: DataFrame spark
ds = spark.read.csv(path='Advertising.csv',
#           sep=', ',
#           encoding='UTF-8',
#           comment=None,
#           header=True,
#           inferSchema=True)
```

3. From .json

Data from: <http://api.luftdaten.info/static/v1/data.json>

```
dp = pd.read_json("data/data.json")
ds = spark.read.json('data/data.json')
```

:: Python Code:

```
dp[['id', 'timestamp']].head(4)
#
ds[['id', 'timestamp']].show(4)
```

:: Comparison:

↪--+					+-----+-----+-----+-----+
↪timestamp					id
	id	timestamp			+-----+-----+-----+-----+
↪--+					
0	2994551481	2019-02-28 17:23:52			2994551481 2019-02-28_
↪17:23:52					
1	2994551482	2019-02-28 17:23:52			2994551482 2019-02-28_
↪17:23:52					
2	2994551483	2019-02-28 17:23:52			2994551483 2019-02-28_
↪17:23:52					
3	2994551484	2019-02-28 17:23:52			2994551484 2019-02-28_
↪17:23:52					
↪--+					+-----+-----+-----+-----+
					only showing top 4 rows

5.3.3 First n Rows

:: Python Code:

```
dp.head(4)
#
ds.show(4)
```

:: Comparison:

					+-----+-----+-----+-----+
					TV Radio Newspaper Sales
					+-----+-----+-----+-----+
0	TV	Radio	Newspaper	Sales	230.1 37.8 69.2 22.1
1	44.5	39.3	45.1	10.4	44.5 39.3 45.1 10.4
2	17.2	45.9	69.3	9.3	17.2 45.9 69.3 9.3
3	151.5	41.3	58.5	18.5	151.5 41.3 58.5 18.5
					+-----+-----+-----+-----+
					only showing top 4 rows

5.3.4 Column Names

:: Python Code:

```
dp.columns
#
ds.columns
```

:: Comparison:

```
Index(['TV', 'Radio', 'Newspaper', 'Sales'], dtype='object')
['TV', 'Radio', 'Newspaper', 'Sales']
```

5.3.5 Data types

:: Python Code:

```
dp.dtypes
#
ds.dtypes
```

:: Comparison:

TV	float64	(('TV', 'double'),
Radio	float64	('Radio', 'double'),
Newspaper	float64	('Newspaper', 'double'),
Sales	float64	('Sales', 'double')]
dtype:	object	

5.3.6 Fill Null

```
my_list = [['male', 1, None], ['female', 2, 3], ['male', 3, 4]]
dp = pd.DataFrame(my_list, columns=['A', 'B', 'C'])
ds = spark.createDataFrame(my_list, ['A', 'B', 'C'])
#
dp.head()
ds.show()
```

:: Comparison:

				+-----+-----+-----+
				A B C
				+-----+-----+-----+
0	male	1	NaN	male 1 null
1	female	2	3.0	female 2 3
2	male	3	4.0	male 3 4
				+-----+-----+-----+

:: Python Code:

```
dp.fillna(-99)
#
ds.fillna(-99).show()
```

:: Comparison:

				+-----+-----+-----+
				A B C

(continues on next page)

(continued from previous page)

	A	B	C	
0	male	1	-99	male 1 -99
1	female	2	3.0	female 2 3
2	male	3	4.0	male 3 4

5.3.7 Replace Values

:: Python Code:

```
# caution: you need to chose specific col
dp.A.replace(['male', 'female'], [1, 0], inplace=True)
dp
#caution: Mixed type replacements are not supported
ds.na.replace(['male', 'female'], ['1', '0']).show()
```

:: Comparison:

	A	B	C	
0	1	1	NaN	1 1 null
1	0	2	3.0	0 2 3
2	1	3	4.0	1 3 4

5.3.8 Rename Columns

1. Rename all columns

:: Python Code:

```
dp.columns = ['a', 'b', 'c', 'd']
dp.head(4)
#
ds.toDF('a', 'b', 'c', 'd').show(4)
```

:: Comparison:

	a	b	c	d	
0	230.1	37.8	69.2	22.1	230.1 37.8 69.2 22.1
1	44.5	39.3	45.1	10.4	44.5 39.3 45.1 10.4
2	17.2	45.9	69.3	9.3	17.2 45.9 69.3 9.3
3	151.5	41.3	58.5	18.5	151.5 41.3 58.5 18.5

only showing top 4 rows

2. Rename one or more columns

```
mapping = {'Newspaper':'C', 'Sales':'D'}
```

:: Python Code:

```
dp.rename(columns=mapping).head(4)
#
new_names = [mapping.get(col,col) for col in ds.columns]
ds.toDF(*new_names).show(4)
```

:: Comparison:

	TV	Radio	C	D
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5

only showing top 4 rows

Note: You can also use `withColumnRenamed` to rename one column in PySpark.

:: Python Code:

```
ds.withColumnRenamed('Newspaper', 'Paper').show(4)
```

:: Comparison:

	TV	Radio	Paper	Sales
	230.1	37.8	69.2	22.1
	44.5	39.3	45.1	10.4
	17.2	45.9	69.3	9.3
	151.5	41.3	58.5	18.5

only showing top 4 rows

5.3.9 Drop Columns

```
drop_name = ['Newspaper', 'Sales']
```

:: Python Code:

```
dp.drop(drop_name,axis=1).head(4)
#
ds.drop(*drop_name).show(4)
```

:: Comparison:

	TV	Radio
0	230.1	37.8
1	44.5	39.3
2	17.2	45.9
3	151.5	41.3

	TV	Radio
0	230.1	37.8
1	44.5	39.3
2	17.2	45.9
3	151.5	41.3

only showing top 4 rows

5.3.10 Filter

```
dp = pd.read_csv('Advertising.csv')
#
ds = spark.read.csv(path='Advertising.csv',
                    header=True,
                    inferSchema=True)
```

:: Python Code:

```
dp[dp.Newspaper<20].head(4)
#
ds[ds.Newspaper<20].show(4)
```

:: Comparison:

	TV	Radio	Newspaper	Sales
7	120.2	19.6	11.6	13.2
8	8.6	2.1	1.0	4.8
11	214.7	24.0	4.0	17.4
13	97.5	7.6	7.2	9.7

	TV	Radio	Newspaper	Sales
7	120.2	19.6	11.6	13.2
8	8.6	2.1	1.0	4.8
11	214.7	24.0	4.0	17.4
13	97.5	7.6	7.2	9.7

only showing top 4 rows

:: Python Code:

```
dp[(dp.Newspaper<20) & (dp.TV>100)].head(4)
#
ds[(ds.Newspaper<20) & (ds.TV>100)].show(4)
```

:: Comparison:

	TV	Radio	Newspaper	Sales	
7	120.2	19.6	11.6	13.2	120.2 19.6 11.6 13.2
11	214.7	24.0	4.0	17.4	214.7 24.0 4.0 17.4
19	147.3	23.9	19.1	14.6	147.3 23.9 19.1 14.6
25	262.9	3.5	19.5	12.0	262.9 3.5 19.5 12.0

only showing top 4 rows

5.3.11 With New Column

:: Python Code:

```
dp['tv_norm'] = dp.TV/sum(dp.TV)
dp.head(4)
#
ds.withColumn('tv_norm', ds.TV/ds.groupBy().agg(F.sum("TV")).collect()[0][0]).
  show(4)
```

:: Comparison:

	TV	Radio	Newspaper	Sales	tv_norm	
0	230.1	37.8	69.2	22.1	0.007824	230.1 37.8 69.2 22.1 0.007824
1	44.5	39.3	45.1	10.4	0.001513	44.5 39.3 45.1 10.4 0.001513
2	17.2	45.9	69.3	9.3	0.000585	17.2 45.9 69.3 9.3 0.000585
3	151.5	41.3	58.5	18.5	0.005152	151.5 41.3 58.5 18.5 0.005152

only showing top 4 rows

:: Python Code:

```
dp['cond'] = dp.apply(lambda c: 1 if ((c.TV>100)&(c.Radio<40)) else 2 if c.
    ↳Sales> 10 else 3,axis=1)
#
ds.withColumn('cond',F.when((ds.TV>100)&(ds.Radio<40),1)\
    .when(ds.Sales>10, 2)\
    .otherwise(3)).show(4)
```

:: Comparison:

	TV	Radio	Newspaper	Sales	cond
0	230.1	37.8	69.2	22.1	1
1	44.5	39.3	45.1	10.4	2
2	17.2	45.9	69.3	9.3	3
3	151.5	41.3	58.5	18.5	2

only showing top 4 rows

:: Python Code:

```
dp['log_tv'] = np.log(dp.TV)
dp.head(4)
#
import pyspark.sql.functions as F
ds.withColumn('log_tv',F.log(ds.TV)).show(4)
```

:: Comparison:

	TV	Radio	Newspaper	Sales	log_tv
0	230.1	37.8	69.2	22.1	5.438514
1	44.5	39.3	45.1	10.4	3.795489
2	17.2	45.9	69.3	9.3	2.844909
3	151.5	41.3	58.5	18.5	5.020586

(continues on next page)

(continued from previous page)

only showing top 4 rows

:: Python Code:

```
dp['tv+10'] = dp.TV.apply(lambda x: x+10)
dp.head(4)
#
ds.withColumn('tv+10', ds.TV+10).show(4)
```

:: Comparison:

```

↪-----+
|
↪TV|Radio|Newspaper|Sales|tv+10|
   TV  Radio  Newspaper  Sales  tv+10
↪-----+
0  230.1   37.8         69.2   22.1  240.1 |230.1| 37.8|         69.2| 22.
↪1|240.1|
1   44.5   39.3         45.1   10.4   54.5 | 44.5| 39.3|         45.1| 10.4|
↪54.5|
2   17.2   45.9         69.3    9.3   27.2 | 17.2| 45.9|         69.3|  9.3|
↪27.2|
3  151.5   41.3         58.5   18.5  161.5 |151.5| 41.3|         58.5| 18.
↪5|161.5|
|
↪-----+
only showing top 4 rows
```

5.3.12 Join

```
leftp = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                      'B': ['B0', 'B1', 'B2', 'B3'],
                      'C': ['C0', 'C1', 'C2', 'C3'],
                      'D': ['D0', 'D1', 'D2', 'D3']},
                      index=[0, 1, 2, 3])

rightp = pd.DataFrame({'A': ['A0', 'A1', 'A6', 'A7'],
                       'F': ['B4', 'B5', 'B6', 'B7'],
                       'G': ['C4', 'C5', 'C6', 'C7'],
                       'H': ['D4', 'D5', 'D6', 'D7']},
                       index=[4, 5, 6, 7])

lefts = spark.createDataFrame(leftp)
rights = spark.createDataFrame(rightp)
```

	A	B	C	D		A	F	G	H
0	A0	B0	C0	D0	4	A0	B4	C4	D4
1	A1	B1	C1	D1	5	A1	B5	C5	D5

(continues on next page)

(continued from previous page)

2	A2	B2	C2	D2	6	A6	B6	C6	D6
3	A3	B3	C3	D3	7	A7	B7	C7	D7

1. Left Join

:: Python Code:

```
lefttp.merge(righttp,on='A',how='left')
#
leftts.join(rights,on='A',how='left')
.orderBy('A',ascending=True).show()
```

:: Comparison:

	A	B	C	D	F	G	H
0	A0	B0	C0	D0	B4	C4	D4
1	A1	B1	C1	D1	B5	C5	D5
2	A2	B2	C2	D2	NaN	NaN	NaN
3	A3	B3	C3	D3	NaN	NaN	NaN

2. Right Join

:: Python Code:

```
lefttp.merge(righttp,on='A',how='right')
#
leftts.join(rights,on='A',how='right')
.orderBy('A',ascending=True).show()
```

:: Comparison:

	A	B	C	D	F	G	H
0	A0	B0	C0	D0	B4	C4	D4
1	A1	B1	C1	D1	B5	C5	D5

(continues on next page)

(continued from previous page)

```

2  A6  NaN  NaN  NaN  B6  C6  D6      | A6|null|null|null| B6|
↪C6| D6|
3  A7  NaN  NaN  NaN  B7  C7  D7      | A7|null|null|null| B7|
↪C7| D7|
                                     +---+---+---+---+---+
↪--+-+---+

```

3. Inner Join

:: Python Code:

```

lefttp.merge(righttp,on='A',how='inner')
#
lefts.join(rights,on='A',how='inner')
      .orderBy('A',ascending=True).show()

```

:: Comparison:

```

                                     +---+---+---+---+---+---+
                                     |  A|  B|  C|  D|  F|  G|  H|
0  A0  B0  C0  D0  B4  C4  D4      | A0| B0| C0| D0| B4| C4| D4|
1  A1  B1  C1  D1  B5  C5  D5      | A1| B1| C1| D1| B5| C5| D5|
                                     +---+---+---+---+---+---+

```

4. Full Join

:: Python Code:

```

lefttp.merge(righttp,on='A',how='outer')
#
lefts.join(rights,on='A',how='full')
      .orderBy('A',ascending=True).show()

```

:: Comparison:

```

↪+---+---+
                                     +---+---+---+---+---+
                                     |  A|  B|  C|  D|
↪F|  G|  H|
    A    B    C    D    F    G    H      | A0| B0| C0| D0|
↪B4|  C4|  D4|
1  A1  B1  C1  D1  B5  C5  D5      | A1| B1| C1| D1|
↪B5|  C5|  D5|
2  A2  B2  C2  D2  NaN  NaN  NaN      | A2| B2| C2|
↪D2|null|null|null|
3  A3  B3  C3  D3  NaN  NaN  NaN      | A3| B3| C3|
↪D3|null|null|null|
4  A6  NaN  NaN  NaN  B6  C6  D6      | A6|null|null|null|
↪B6|  C6|  D6|

```

(continues on next page)

(continued from previous page)

```

5  A7  NaN  NaN  NaN  B7  C7  D7  | A7|null|null|null|
↪B7|  C7|  D7|
                                     +---+---+---+---+---+
↪+---+---+

```

5.3.13 Concat Columns

```

my_list = [('a', 2, 3),
           ('b', 5, 6),
           ('c', 8, 9),
           ('a', 2, 3),
           ('b', 5, 6),
           ('c', 8, 9)]
col_name = ['col1', 'col2', 'col3']
#
dp = pd.DataFrame(my_list, columns=col_name)
ds = spark.createDataFrame(my_list, schema=col_name)

```

	col1	col2	col3
0	a	2	3
1	b	5	6
2	c	8	9
3	a	2	3
4	b	5	6
5	c	8	9

:: Python Code:

```

dp['concat'] = dp.apply(lambda x: '%s%s'%(x['col1'],x['col2']),axis=1)
dp
#
ds.withColumn('concat',F.concat('col1','col2')).show()

```

:: Comparison:

	col1	col2	col3	concat
0	a	2	3	a2
1	b	5	6	b5
2	c	8	9	c8
3	a	2	3	a2
4	b	5	6	b5
5	c	8	9	c8

	col1	col2	col3	concat
0	a	2	3	a2
1	b	5	6	b5
2	c	8	9	c8
3	a	2	3	a2
4	b	5	6	b5
5	c	8	9	c8

5.3.14 GroupBy

:: Python Code:

```
dp.groupby(['col1']).agg({'col2':'min','col3':'mean'})
#
ds.groupBy(['col1']).agg({'col2': 'min', 'col3': 'avg'}).show()
```

:: Comparison:

	col2	col3																	
col1			<table> <tr> <th></th><th>col1</th><th>min(col2)</th><th>avg(col3)</th></tr> <tr> <td>a</td><td>c</td><td>8</td><td>9.0</td></tr> <tr> <td>b</td><td>b</td><td>5</td><td>6.0</td></tr> <tr> <td>c</td><td>a</td><td>2</td><td>3.0</td></tr> </table>		col1	min(col2)	avg(col3)	a	c	8	9.0	b	b	5	6.0	c	a	2	3.0
	col1	min(col2)	avg(col3)																
a	c	8	9.0																
b	b	5	6.0																
c	a	2	3.0																
a	2	3																	
b	5	6																	
c	8	9																	

5.3.15 Pivot

:: Python Code:

```
pd.pivot_table(dp, values='col3', index='col1', columns='col2', aggfunc=np.
    ↪sum)
#
ds.groupBy(['col1']).pivot('col2').sum('col3').show()
```

:: Comparison:

col2	2	5	8																					
col1				<table> <tr> <th></th><th>col1</th><th>2</th><th>5</th><th>8</th></tr> <tr> <td>a</td><td>c</td><td>null</td><td>null</td><td>18</td></tr> <tr> <td>b</td><td>b</td><td>null</td><td>12</td><td>null</td></tr> <tr> <td>c</td><td>a</td><td>6</td><td>null</td><td>null</td></tr> </table>		col1	2	5	8	a	c	null	null	18	b	b	null	12	null	c	a	6	null	null
	col1	2	5	8																				
a	c	null	null	18																				
b	b	null	12	null																				
c	a	6	null	null																				
a	6.0	NaN	NaN																					
b	NaN	12.0	NaN																					
c	NaN	NaN	18.0																					

5.3.16 Window

```
d = {'A':['a','b','c','d'],'B':['m','m','n','n'],'C':[1,2,3,6]}
dp = pd.DataFrame(d)
ds = spark.createDataFrame(dp)
```

:: Python Code:

```
dp['rank'] = dp.groupby('B')['C'].rank('dense',ascending=False)
#
from pyspark.sql.window import Window
```

(continues on next page)

(continued from previous page)

```
w = Window.partitionBy('B').orderBy(ds.C.desc())
ds = ds.withColumn('rank',F.rank().over(w))
```

:: Comparison:

	A	B	C	rank
0	a	m	1	2.0
1	b	m	2	1.0
2	c	n	3	2.0
3	d	n	6	1.0

5.3.17 rank VS dense_rank

```
d = {'Id': [1, 2, 3, 4, 5, 6],
      'Score': [4.00, 4.00, 3.85, 3.65, 3.65, 3.50]}
#
data = pd.DataFrame(d)
dp = data.copy()
ds = spark.createDataFrame(data)
```

	Id	Score
0	1	4.00
1	2	4.00
2	3	3.85
3	4	3.65
4	5	3.65
5	6	3.50

:: Python Code:

```
dp['Rank_dense'] = dp['Score'].rank(method='dense',ascending =False)
dp['Rank'] = dp['Score'].rank(method='min',ascending =False)
dp
#
import pyspark.sql.functions as F
from pyspark.sql.window import Window
w = Window.orderBy(ds.Score.desc())
ds = ds.withColumn('Rank_spark_dense',F.dense_rank().over(w))
ds = ds.withColumn('Rank_spark',F.rank().over(w))
ds.show()
```

:: Comparison:

	Id	Score	Rank_dense	Rank
0	1	4.00	1	1
1	2	4.00	1	2
2	3	3.85	2	3
3	4	3.65	3	4
4	5	3.65	3	5
5	6	3.50	4	6

(continues on next page)

(continued from previous page)

0	1	4.00	1.0	1.0		1	4.0	1	1
1	2	4.00	1.0	1.0		2	4.0	1	1
2	3	3.85	2.0	3.0		3	3.85	2	3
3	4	3.65	3.0	4.0		4	3.65	3	4
4	5	3.65	3.0	4.0		5	3.65	3	4
5	6	3.50	4.0	6.0		6	3.5	4	6
+---+-----+-----+-----+									

STATISTICS AND LINEAR ALGEBRA PRELIMINARIES

Chinese proverb

If you only know yourself, but not your opponent, you may win or may lose. If you know neither yourself nor your enemy, you will always endanger yourself – idiom, from Sunzi's Art of War

6.1 Notations

- m : the number of the samples
- n : the number of the features
- y_i : i -th label
- $\hat{y}_i$: i -th predicted label
- $\bar{\mathbf{y}} = \frac{1}{m} \sum_{i=1}^m y_i$: the mean of $\mathbf{y}$.
- $\mathbf{y}$: the label vector.
- $\hat{\mathbf{y}}$: the predicted label vector.

6.2 Linear Algebra Preliminaries

Since I have documented the Linear Algebra Preliminaries in my Prelim Exam note for Numerical Analysis, the interested reader is referred to [Feng2014] for more details (Figure. [Linear Algebra Preliminaries](#)).

1 Preliminaries

1.1 Linear Algebra Preliminaries

1.1.1 Common Properties

Properties 1.1. (Structure of Matrices) Let $A = [A_{ij}]$ be a square or rectangular matrix, A is called	
• <i>diagonal</i> : if $a_{ij} = 0, \forall i \neq j$, • <i>upper triangular</i> : if $a_{ij} = 0, \forall i > j$, • <i>upper Hessenberg</i> : if $a_{ij} = 0, \forall i > j + 1$, • <i>block diagonal</i> : $A = \text{diag}(A_{11}, A_{22}, \dots, A_{nn})$,	• <i>tridiagonal</i> : if $a_{ij} = 0, \forall i - j > 1$, • <i>lower triangular</i> : if $a_{ij} = 0, \forall i < j$, • <i>lower Hessenberg</i> : if $a_{ij} = 0, \forall j > i + 1$, • <i>block diagonal</i> : $A = \text{diag}(A_{i_{i-1}}, A_{ii}, \dots, A_{i_{i+1}})$.
Properties 1.2. (Type of Matrices) Let $A = [A_{ij}]$ be a square or rectangular matrix, A is called	
• <i>Hermitian</i> : if $A^* = A$, • <i>symmetric</i> : if $A^T = A$, • <i>normal</i> : if $A^T A = A A^T$, when $A \in \mathbb{R}^{n \times n}$, if $A^* A = A A^*$, when $A \in \mathbb{C}^{n \times n}$,	• <i>skew hermitian</i> : if $A^* = -A$, • <i>skew symmetric</i> : if $A^T = -A$, • <i>orthogonal</i> : if $A^T A = I$, when $A \in \mathbb{R}^{n \times n}$, • <i>unitary</i> : if $A^* A = I$, when $A \in \mathbb{C}^{n \times n}$.
Properties 1.3. (Properties of invertible matrices) Let A be $n \times n$ square matrix. If A is <i>invertible</i> , then	
• $\det(A) \neq 0$, • $\text{rank}(A) = n$, • $Ax = b$ has a unique solution for every $b \in \mathbb{R}^n$ • the row vectors are <i>linearly independent</i>, • the row vectors of A form a basis for $\mathbb{R}^n$. • the row vectors of A span $\mathbb{R}^n$.	• $\text{nullity}(A) = 0$, • $\lambda_i \neq 0, (\lambda_i \text{ eigenvalues})$, • $Ax = 0$ has only trivial solution, • the column vectors are <i>linearly independent</i>, • the column vectors of A form a basis for $\mathbb{R}^n$, • the column vectors of A span $\mathbb{R}^n$.
Properties 1.4. (Properties of conjugate transpose) Let A, B be $n \times n$ square matrix and γ be a complex constant, then	
• $(A^*)^* = A$, • $(AB)^* = B^* A^*$, • $(A + B)^* = A^* + B^*$,	• $\det(A^*) = \det(A)$ • $\text{tr}(A^*) = \text{tr}(A)$ • $(\gamma A)^* = \gamma^* A^*$.
Properties 1.5. (Properties of similar matrices) If $A \sim B$, then	
• $\det(A) = \det(B)$, • $\text{eig}(A) = \text{eig}(B)$, • $A \sim A$,	• $\text{rank}(A) = \text{rank}(B)$, • if $B \sim C$, then $A \sim C$ • $B \sim A$

Fig. 1: Linear Algebra Preliminaries

6.3 Measurement Formula

6.3.1 Mean absolute error

In statistics, **MAE** (Mean absolute error) is a measure of difference between two continuous variables. The Mean Absolute Error is given by:

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|.$$

6.3.2 Mean squared error

In statistics, the **MSE** (Mean Squared Error) of an estimator (of a procedure for estimating an unobserved quantity) measures the average of the squares of the errors or deviations—that is, the difference between the estimator and what is estimated.

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

6.3.3 Root Mean squared error

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2}$$

6.3.4 Total sum of squares

In statistical data analysis the **TSS** (Total Sum of Squares) is a quantity that appears as part of a standard way of presenting results of such analyses. It is defined as being the sum, over all observations, of the squared differences of each observation from the overall mean.

$$\text{TSS} = \sum_{i=1}^m (y_i - \bar{y})^2$$

6.3.5 Explained Sum of Squares

In statistics, the **ESS** (Explained sum of squares), alternatively known as the model sum of squares or sum of squares due to regression.

The ESS is the sum of the squares of the differences of the predicted values and the mean value of the response variable which is given by:

$$\text{ESS} = \sum_{i=1}^m (\hat{y}_i - \bar{y})^2$$

6.3.6 Residual Sum of Squares

In statistics, **RSS** (Residual sum of squares), also known as the sum of squared residuals (SSR) or the sum of squared errors of prediction (SSE), is the sum of the squares of residuals which is given by:

$$\text{RSS} = \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

6.3.7 Coefficient of determination R^2

$$R^2 := \frac{ESS}{TSS} = 1 - \frac{\text{RSS}}{\text{TSS}}.$$

Note: In general ($\mathbf{y}^T \bar{\mathbf{y}} = \hat{\mathbf{y}}^T \bar{\mathbf{y}}$), total sum of squares = explained sum of squares + residual sum of squares, i.e.:

$$\text{TSS} = \text{ESS} + \text{RSS} \text{ if and only if } \mathbf{y}^T \bar{\mathbf{y}} = \hat{\mathbf{y}}^T \bar{\mathbf{y}}.$$

More details can be found at [Partitioning in the general ordinary least squares model](#).

6.4 Confusion Matrix

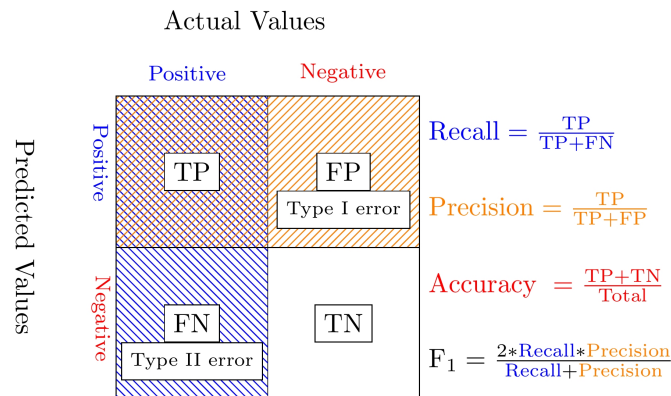


Fig. 2: Confusion Matrix

6.4.1 Recall

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

6.4.2 Precision

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

6.4.3 Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total}}$$

6.4.4 F_1 -score

$$F_1 = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

6.5 Statistical Tests

6.5.1 Correlational Test

- Pearson correlation: Tests for the strength of the association between two continuous variables.
- Spearman correlation: Tests for the strength of the association between two ordinal variables (does not rely on the assumption of normal distributed data).
- Chi-square: Tests for the strength of the association between two categorical variables.

6.5.2 Comparison of Means test

- Paired T-test: Tests for difference between two related variables.
- Independent T-test: Tests for difference between two independent variables.
- ANOVA: Tests the difference between group means after any other variance in the outcome variable is accounted for.

6.5.3 Non-parametric Test

- Wilcoxon rank-sum test: Tests for difference between two independent variables - takes into account magnitude and direction of difference.
- Wilcoxon sign-rank test: Tests for difference between two related variables - takes into account magnitude and direction of difference.
- Sign test: Tests if two related variables are different – ignores magnitude of change, only takes into account direction.

DATA EXPLORATION

Chinese proverb

A journey of a thousand miles begins with a single step – idiom, from Laozi.

I wouldn't say that understanding your dataset is the most difficult thing in data science, but it is really important and time-consuming. Data Exploration is about describing the data by means of statistical and visualization techniques. We explore data in order to understand the features and bring important features to our models.

7.1 Univariate Analysis

In mathematics, univariate refers to an expression, equation, function or polynomial of only one variable. “Uni” means “one”, so in other words your data has only one variable. So you do not need to deal with the causes or relationships in this step. Univariate analysis takes data, summarizes that variables (attributes) one by one and finds patterns in the data.

There are many ways that can describe patterns found in univariate data include central tendency (mean, mode and median) and dispersion: range, variance, maximum, minimum, quartiles (including the interquartile range), coefficient of variation and standard deviation. You also have several options for visualizing and describing data with univariate data. Such as frequency Distribution Tables, bar Charts, histograms, frequency Polygons, pie Charts.

The variable could be either categorical or numerical, I will demonstrate the different statistical and visualization techniques to investigate each type of the variable.

- The Jupyter notebook can be download from [Data Exploration](#).
- The data can be download from [German Credit](#).

7.1.1 Numerical Variables

Describe

The describe function in pandas and spark will give us most of the statistical results, such as min, median, max, quartiles and standard deviation. With the help of the user defined function, you can get even more statistical results.

```
# selected variables for the demonstration
num_cols = ['Account Balance', 'No of dependents']
df.select(num_cols).describe().show()
```

```
+-----+-----+-----+
|summary| Account Balance| No of dependents|
+-----+-----+-----+
| count|           1000|           1000|
|  mean|           2.577|           1.155|
| stddev|1.2576377271108936|0.36208577175319395|
|   min|              1|              1|
|   max|              4|              2|
+-----+-----+-----+
```

You may find out that the default function in PySpark does not include the quartiles. The following function will help you to get the same results in Pandas

```
def describe_pd(df_in, columns, deciles=False):
    """
    Function to union the basic stats results and deciles
    :param df_in: the input dataframe
    :param columns: the cloumn name list of the numerical variable
    :param deciles: the deciles output

    :return : the numerical describe info. of the input dataframe

    :author: Ming Chen and Wenqiang Feng
    :email: von198@gmail.com
    """

    if deciles:
        percentiles = np.array(range(0, 110, 10))
    else:
        percentiles = [25, 50, 75]

    percs = np.transpose([np.percentile(df_in.select(x).collect(), p,
    ↪percentiles) for x in columns])
    percs = pd.DataFrame(percs, columns=columns)
    percs['summary'] = [str(p) + '%' for p in percentiles]

    spark_describe = df_in.describe().toPandas()
    new_df = pd.concat([spark_describe, percs], ignore_index=True)
    new_df = new_df.round(2)
    return new_df[['summary'] + columns]
```



```
describe_pd(df, num_cols)
```

```
+-----+-----+-----+
|summary| Account Balance| No of dependents|
+-----+-----+-----+
| count |          1000.0|          1000.0|
| mean  |           2.577|           1.155|
| stddev|1.2576377271108936|0.362085771753194|
|  min  |            1.0|            1.0|
|  max  |            4.0|            2.0|
|  25%  |            1.0|            1.0|
|  50%  |            2.0|            1.0|
|  75%  |            4.0|            1.0|
+-----+-----+-----+
```

Sometimes, because of the confidential data issues, you can not deliver the real data and your clients may ask more statistical results, such as deciles. You can apply the following function to achieve it.

```
describe_pd(df, num_cols, deciles=True)
```

```
+-----+-----+-----+
|summary| Account Balance| No of dependents|
+-----+-----+-----+
| count |          1000.0|          1000.0|
| mean  |           2.577|           1.155|
| stddev|1.2576377271108936|0.362085771753194|
|  min  |            1.0|            1.0|
|  max  |            4.0|            2.0|
|   0%  |            1.0|            1.0|
|  10%  |            1.0|            1.0|
|  20%  |            1.0|            1.0|
|  30%  |            2.0|            1.0|
|  40%  |            2.0|            1.0|
|  50%  |            2.0|            1.0|
|  60%  |            3.0|            1.0|
|  70%  |            4.0|            1.0|
|  80%  |            4.0|            1.0|
|  90%  |            4.0|            2.0|
| 100%  |            4.0|            2.0|
+-----+-----+-----+
```

Skewness and Kurtosis

This subsection comes from Wikipedia [Skewness](#).

In probability theory and statistics, skewness is a measure of the asymmetry of the probability distribution of a real-valued random variable about its mean. The skewness value can be positive or negative, or undefined. For a unimodal distribution, negative skew commonly indicates that the tail is on the left side of the distribution, and positive skew indicates that the tail is on the right.

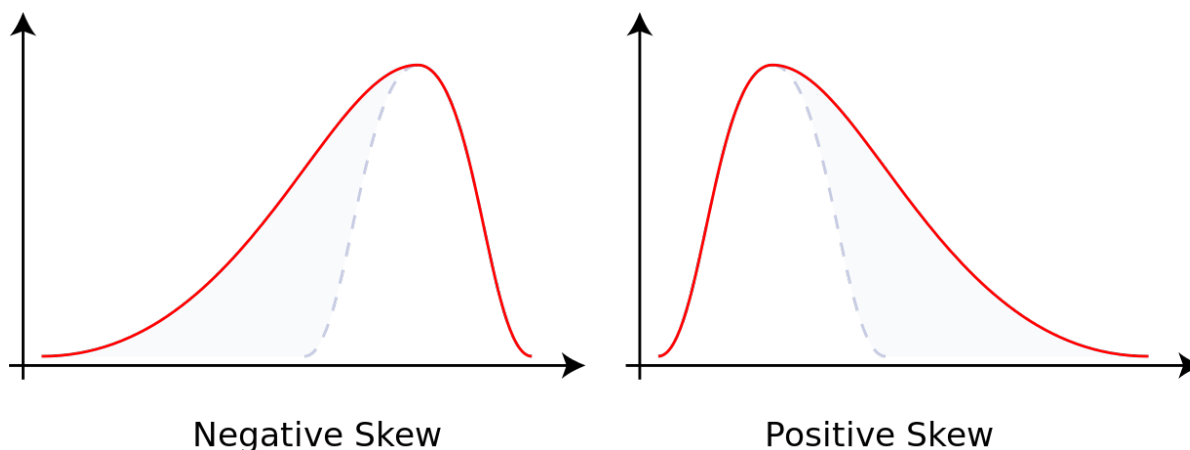
Consider the two distributions in the figure just below. Within each graph, the values on the right side of the

distribution taper differently from the values on the left side. These tapering sides are called tails, and they provide a visual means to determine which of the two kinds of skewness a distribution has:

1. negative skew: The left tail is longer; the mass of the distribution is concentrated on the right of the figure. The distribution is said to be left-skewed, left-tailed, or skewed to the left, despite the fact that the curve itself appears to be skewed or leaning to the right; left instead refers to the left tail being drawn out and, often, the mean being skewed to the left of a typical center of the data. A left-skewed distribution usually appears as a right-leaning curve.
2. positive skew: The right tail is longer; the mass of the distribution is concentrated on the left of the figure. The distribution is said to be right-skewed, right-tailed, or skewed to the right, despite the fact that the curve itself appears to be skewed or leaning to the left; right instead refers to the right tail being drawn out and, often, the mean being skewed to the right of a typical center of the data. A right-skewed distribution usually appears as a left-leaning curve.

This subsection comes from Wikipedia [Kurtosis](#).

In probability theory and statistics, kurtosis (kyrtos or kurtos, meaning “curved, arching”) is a measure of the “tailedness” of the probability distribution of a real-valued random variable. In a similar way to the concept of skewness, kurtosis is a descriptor of the shape of a probability distribution and, just as for skewness, there are different ways of quantifying it for a theoretical distribution and corresponding ways of estimating it from a sample from a population.



```
from pyspark.sql.functions import col, skewness, kurtosis
df.select(skewness(var), kurtosis(var)).show()
```

```
+-----+-----+
|skewness(Age (years))|kurtosis(Age (years))|
+-----+-----+
| 1.0231743160548064 | 0.6114371688367672 |
+-----+-----+
```

Warning: Sometimes the statistics can be misleading!

F. J. Anscombe once said that make both calculations and graphs. Both sorts of output should be stud-

ied; each will contribute to understanding. These 13 datasets in Figure *Same Stats, Different Graphs* (the Datasaurus, plus 12 others) each have the same summary statistics (x/y mean, x/y standard deviation, and Pearson's correlation) to two decimal places, while being drastically different in appearance. This work describes the technique we developed to create this dataset, and others like it. More details and interesting results can be found in [Same Stats Different Graphs](#).

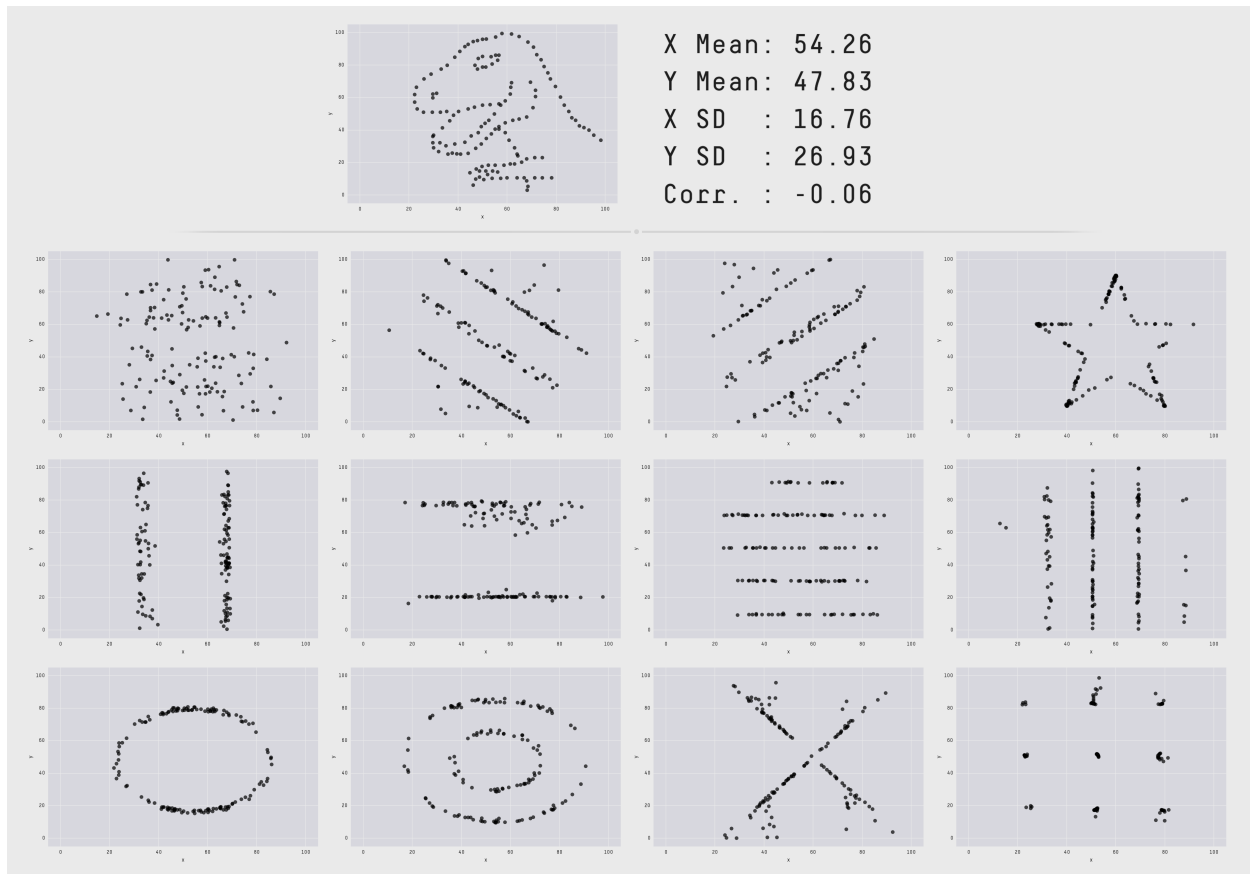


Fig. 1: Same Stats, Different Graphs

Histogram

Warning: Histograms are often confused with Bar graphs!

The fundamental difference between histogram and bar graph will help you to identify the two easily is that there are gaps between bars in a bar graph but in the histogram, the bars are adjacent to each other. The interested reader is referred to [Difference Between Histogram and Bar Graph](#).

```
var = 'Age (years) '  
x = data1[var]  
bins = np.arange(0, 100, 5.0)
```

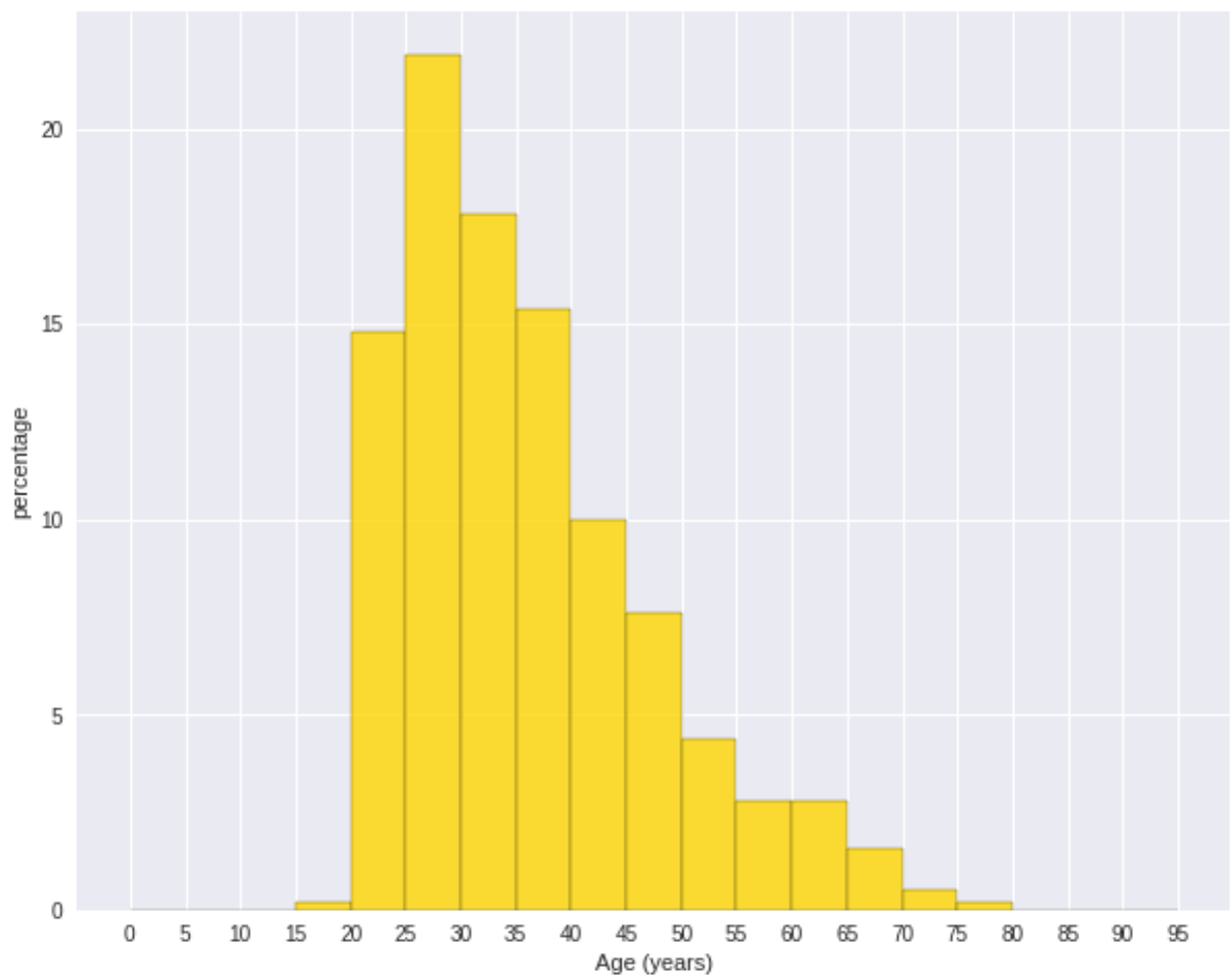
(continues on next page)

(continued from previous page)

```
plt.figure(figsize=(10,8))
# the histogram of the data
plt.hist(x, bins, alpha=0.8, histtype='bar', color='gold',
         ec='black', weights=np.zeros_like(x) + 100. / x.size)

plt.xlabel(var)
plt.ylabel('percentage')
plt.xticks(bins)
plt.show()

fig.savefig(var+".pdf", bbox_inches='tight')
```



```
var = 'Age (years)'
x = data1[var]
bins = np.arange(0, 100, 5.0)

#####
hist, bin_edges = np.histogram(x,bins,
```

(continues on next page)

(continued from previous page)

```

weights=np.zeros_like(x) + 100. / x.size)

# make the histogram

fig = plt.figure(figsize=(20, 8))
ax = fig.add_subplot(1, 2, 1)

# Plot the histogram heights against integers on the x axis
ax.bar(range(len(hist)),hist,width=1,alpha=0.8,ec='black', color='gold')
# # Set the ticks to the middle of the bars
ax.set_xticks([0.5+i for i,j in enumerate(hist)])
# Set the xticklabels to a string that tells us what the bin edges were
labels=['{}'.format(int(bins[i+1])) for i,j in enumerate(hist)]
labels.insert(0,'0')
ax.set_xticklabels(labels)
plt.xlabel(var)
plt.ylabel('percentage')

#####

hist, bin_edges = np.histogram(x,bins) # make the histogram

ax = fig.add_subplot(1, 2, 2)
# Plot the histogram heights against integers on the x axis
ax.bar(range(len(hist)),hist,width=1,alpha=0.8,ec='black', color='gold')

# # Set the ticks to the middle of the bars
ax.set_xticks([0.5+i for i,j in enumerate(hist)])

# Set the xticklabels to a string that tells us what the bin edges were
labels=['{}'.format(int(bins[i+1])) for i,j in enumerate(hist)]
labels.insert(0,'0')
ax.set_xticklabels(labels)
plt.xlabel(var)
plt.ylabel('count')
plt.suptitle('Histogram of {}: Left with percentage output;Right with count_
↪output'
            .format(var), size=16)
plt.show()

fig.savefig(var+".pdf", bbox_inches='tight')

```

Sometimes, some people will ask you to plot the unequal width (invalid argument for histogram) of the bars. You can still achieve it by the following trick.

```

var = 'Credit Amount'
plot_data = df.select(var).toPandas()
x= plot_data[var]

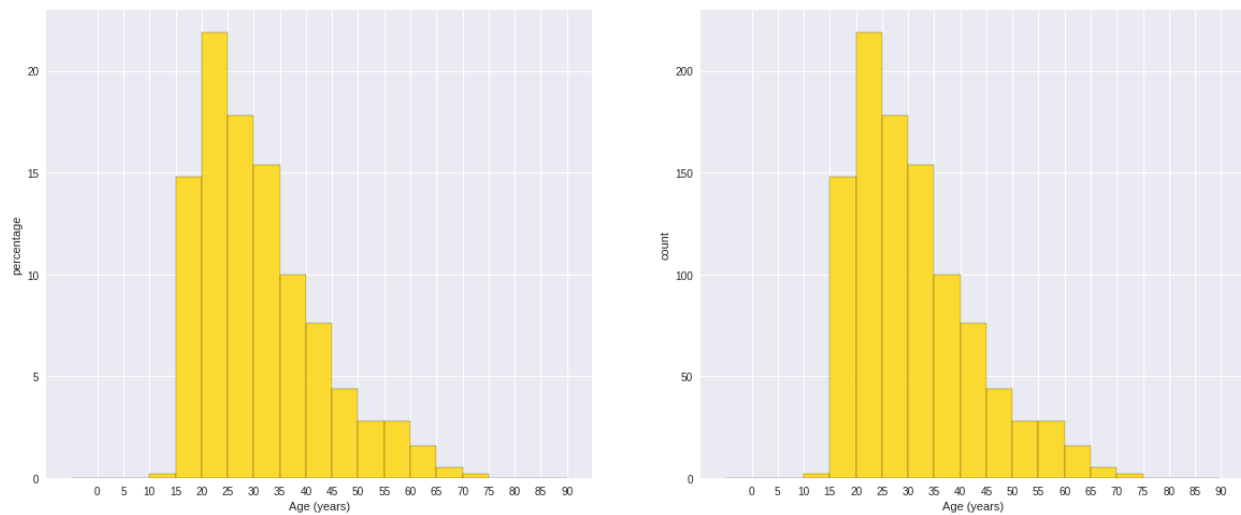
bins =[0,200,400,600,700,800,900,1000,2000,3000,4000,5000,6000,10000,25000]

hist, bin_edges = np.histogram(x,bins,weights=np.zeros_like(x) + 100. / x.
↪size) # make the histogram

```

(continues on next page)

Histogram of Age (years): Left with percentage output; Right with count output



(continued from previous page)

```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(1, 1, 1)
# Plot the histogram heights against integers on the x axis
ax.bar(range(len(hist)), hist, width=1, alpha=0.8, ec='black', color='gold')

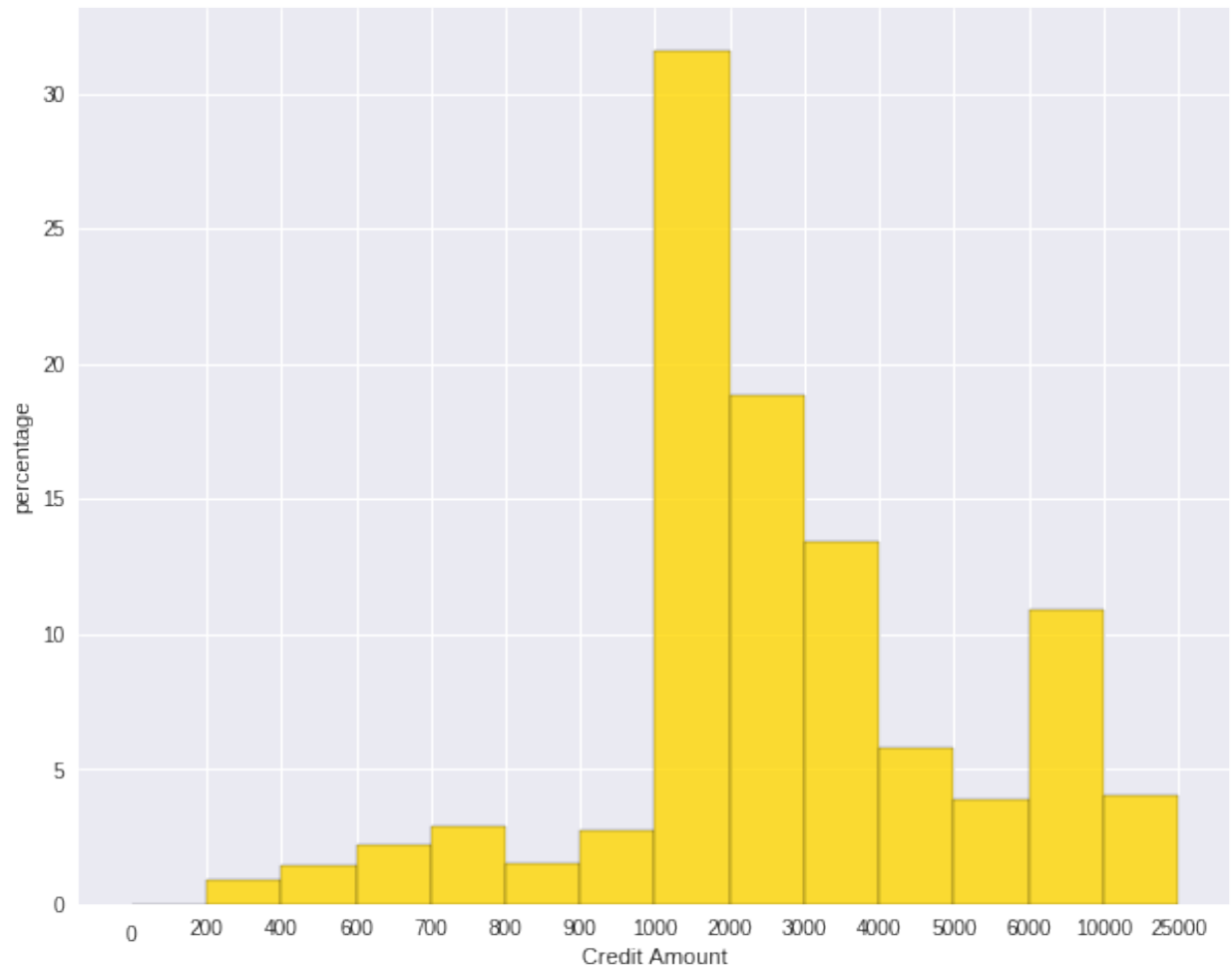
# # Set the ticks to the middle of the bars
ax.set_xticks([0.5+i for i, j in enumerate(hist)])

# Set the xticklabels to a string that tells us what the bin edges were
# labels = ['{}k'.format(int(bins[i+1]/1000)) for i, j in enumerate(hist)]
labels = ['{}'.format(bins[i+1]) for i, j in enumerate(hist)]
labels.insert(0, '0')
ax.set_xticklabels(labels)
# plt.text(-0.6, -1.4, '0')
plt.xlabel(var)
plt.ylabel('percentage')
plt.show()
```

Box plot and violin plot

Note that although violin plots are closely related to Tukey's (1977) box plots, the violin plot can show more information than box plot. When we perform an exploratory analysis, nothing about the samples could be known. So the distribution of the samples can not be assumed to a normal distribution and usually when you get a big data, the normal distribution will show some out liars in box plot.

However, the violin plots are potentially misleading for smaller sample sizes, where the density plots can appear to show interesting features (and group-differences therein) even when produced for standard normal data. Some poster suggested the sample size should larger that 250. The sample sizes (e.g. $n > 250$ or ideally even larger), where the kernel density plots provide a reasonably accurate representation of the distributions,

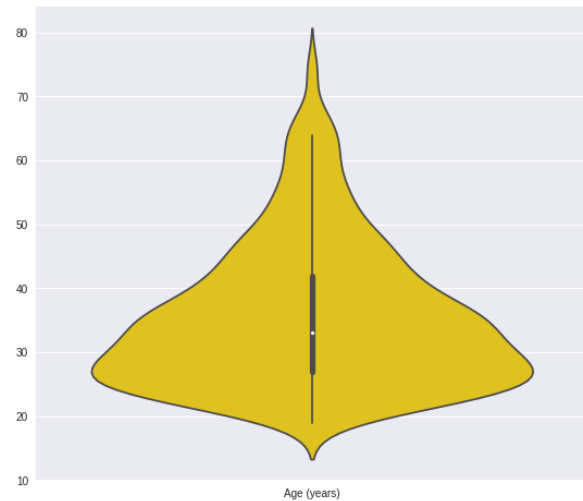
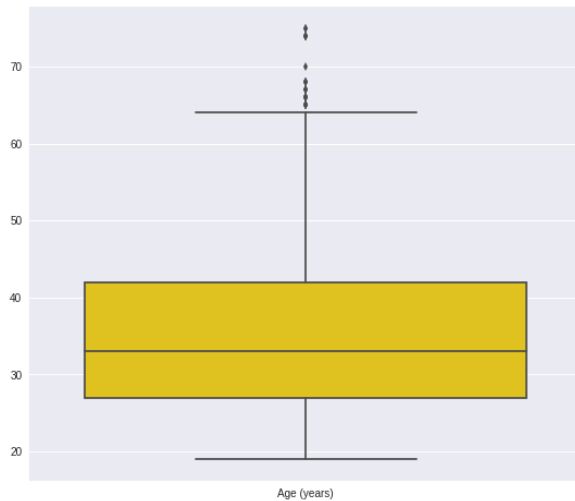


potentially showing nuances such as bimodality or other forms of non-normality that would be invisible or less clear in box plots. More details can be found in [A simple comparison of box plots and violin plots](#).

```
x = df.select(var).toPandas()

fig = plt.figure(figsize=(20, 8))
ax = fig.add_subplot(1, 2, 1)
ax = sns.boxplot(data=x)

ax = fig.add_subplot(1, 2, 2)
ax = sns.violinplot(data=x)
```



7.1.2 Categorical Variables

Compared with the numerical variables, the categorical variables are much more easier to do the exploration.

Frequency table

```
from pyspark.sql import functions as F
from pyspark.sql.functions import rank, sum, col
from pyspark.sql import Window

window = Window.rowsBetween(Window.unboundedPreceding, Window.
    ↳unboundedFollowing)
# withColumn('Percent %', F.format_string("%5.0f%%\n", col('Credit_num') * 100 /
    ↳col('total'))).\
tab = df.select(['age_class', 'Credit Amount']).\
    groupBy('age_class').\
    agg(F.count('Credit Amount').alias('Credit_num'),
        F.mean('Credit Amount').alias('Credit_avg'),
        F.min('Credit Amount').alias('Credit_min'),
        F.max('Credit Amount').alias('Credit_max')).\
```

(continues on next page)

(continued from previous page)

```
withColumn('total',sum(col('Credit_num')).over(window)).\
withColumn('Percent',col('Credit_num')*100/col('total')).\
drop(col('total'))
```

age_class	Credit_num	Credit_avg	Credit_min	Credit_max	Percent
45-54	120	3183.0666666666666	338	12612	12.0
<25	150	2970.7333333333333	276	15672	15.0
55-64	56	3493.660714285714	385	15945	5.6
35-44	254	3403.771653543307	250	15857	25.4
25-34	397	3298.823677581864	343	18424	39.7
65+	23	3210.1739130434785	571	14896	2.3

Pie plot

```
# Data to plot
labels = plot_data.age_class
sizes = plot_data.Percent
colors = ['gold', 'yellowgreen', 'lightcoral', 'blue', 'lightskyblue', 'green',
         ↪ 'red']
explode = (0, 0.1, 0, 0, 0, 0) # explode 1st slice

# Plot
plt.figure(figsize=(10,8))
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=140)

plt.axis('equal')
plt.show()
```

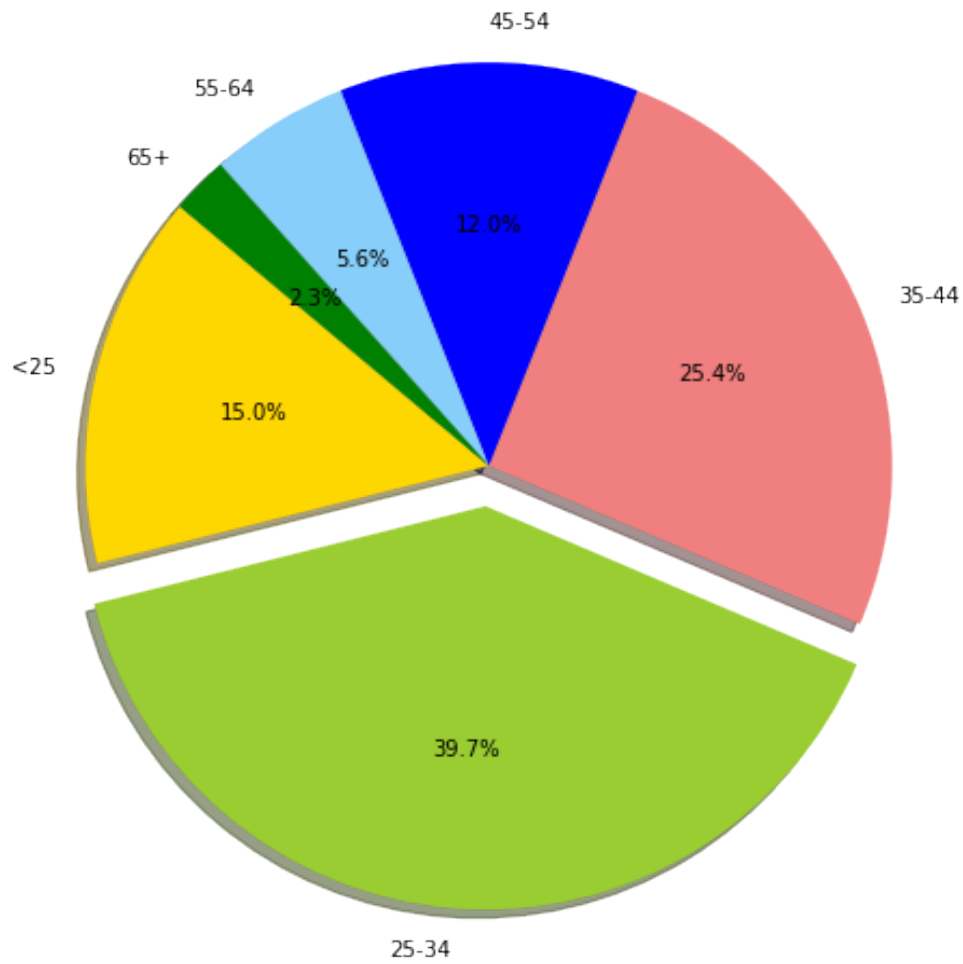
Bar plot

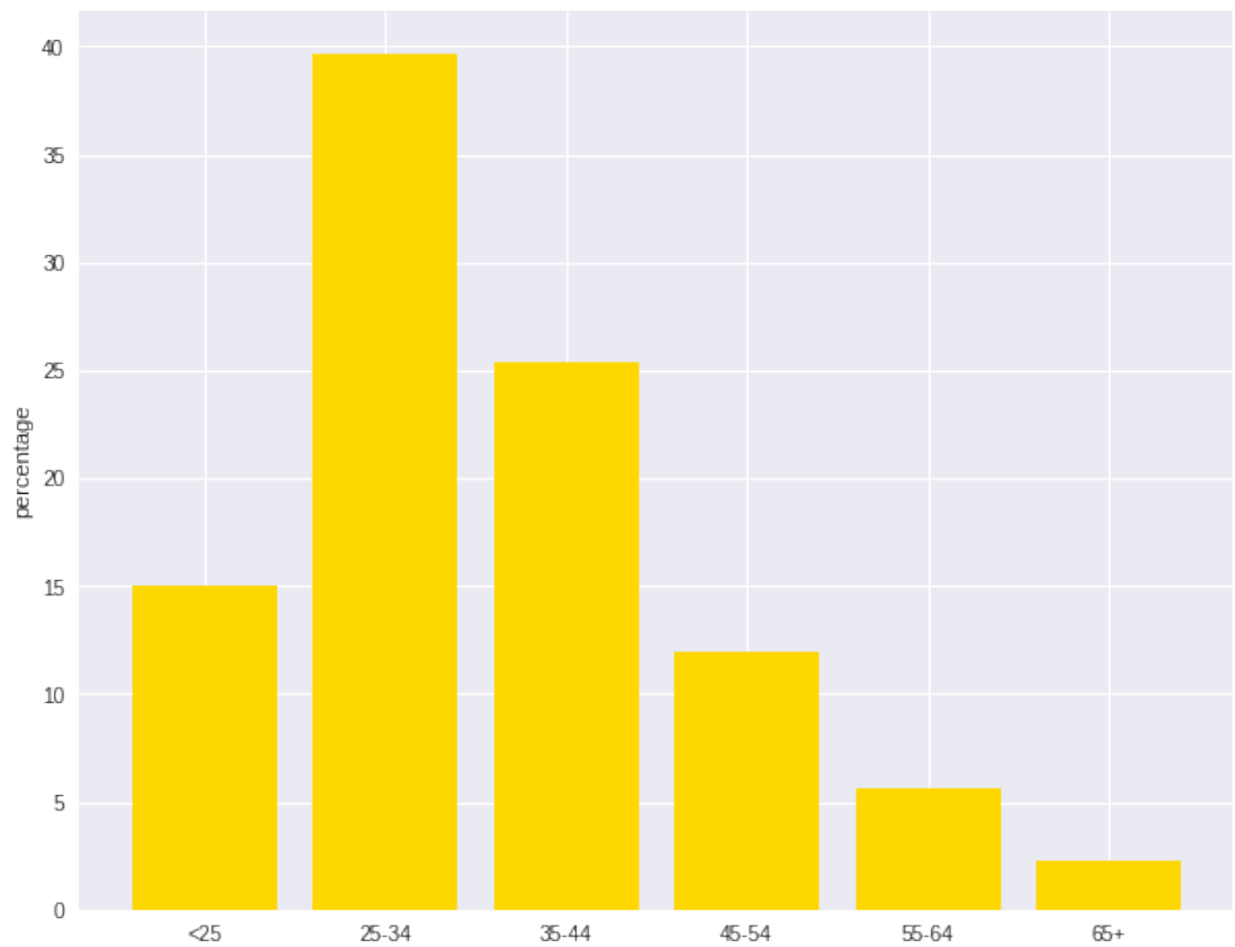
```
labels = plot_data.age_class
missing = plot_data.Percent
ind = [x for x, _ in enumerate(labels)]

plt.figure(figsize=(10,8))
plt.bar(ind, missing, width=0.8, label='missing', color='gold')

plt.xticks(ind, labels)
plt.ylabel("percentage")

plt.show()
```





```

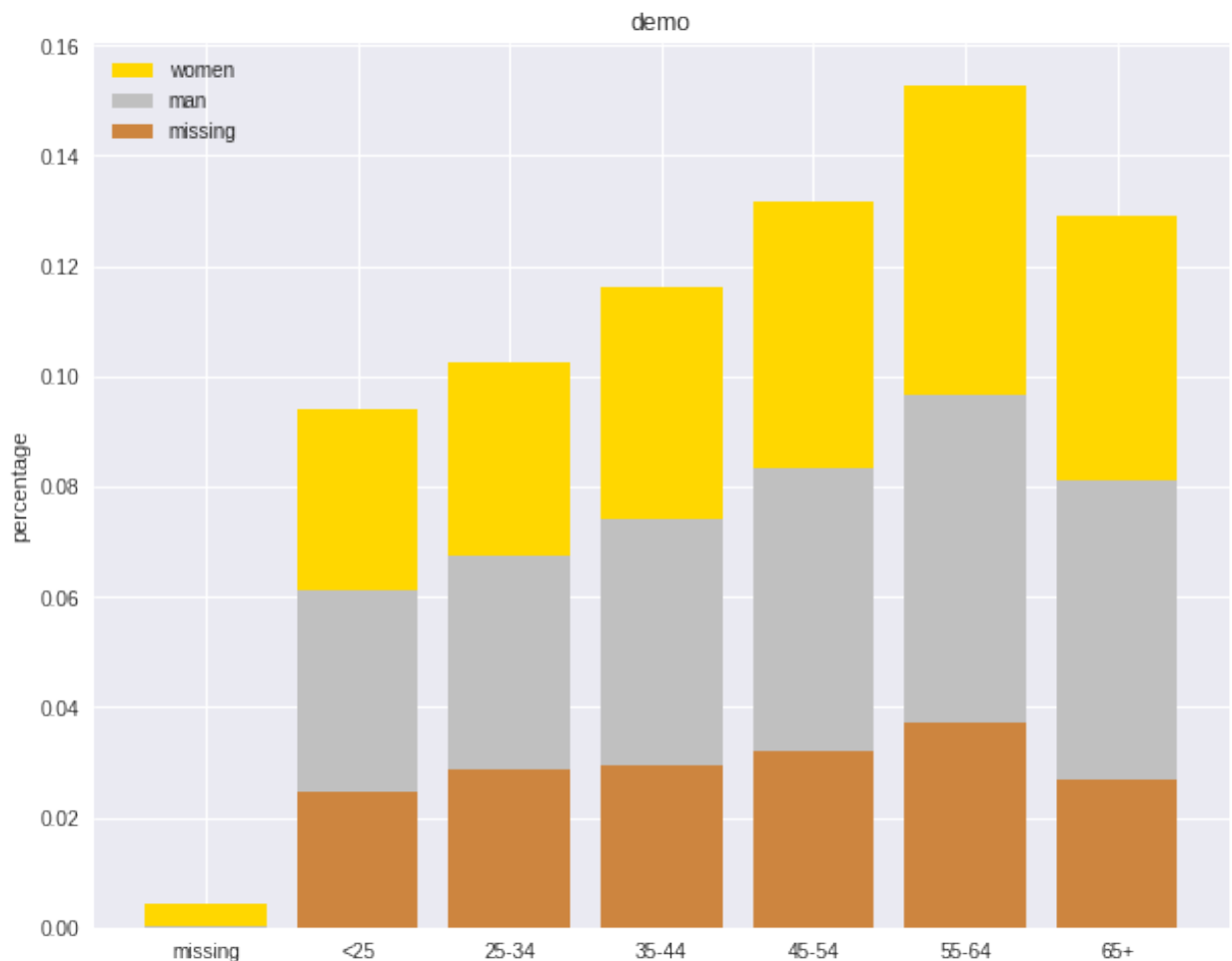
labels = ['missing', '<25', '25-34', '35-44', '45-54', '55-64', '65+']
missing = np.array([0.000095, 0.024830, 0.028665, 0.029477, 0.031918, 0.037073,
    ↪0.026699])
man = np.array([0.000147, 0.036311, 0.038684, 0.044761, 0.051269, 0.059542, 0.
    ↪0.054259])
women = np.array([0.004035, 0.032935, 0.035351, 0.041778, 0.048437, 0.056236,
    ↪0.048091])
ind = [x for x, _ in enumerate(labels)]

plt.figure(figsize=(10,8))
plt.bar(ind, women, width=0.8, label='women', color='gold',
    ↪bottom=man+missing)
plt.bar(ind, man, width=0.8, label='man', color='silver', bottom=missing)
plt.bar(ind, missing, width=0.8, label='missing', color='#CD853F')

plt.xticks(ind, labels)
plt.ylabel("percentage")
plt.legend(loc="upper left")
plt.title("demo")

plt.show()

```



7.2 Multivariate Analysis

In this section, I will only demonstrate the bivariate analysis. Since the multivariate analysis is the generation of the bivariate.

7.2.1 Numerical V.S. Numerical

Correlation matrix

```
from pyspark.mllib.stat import Statistics
import pandas as pd

corr_data = df.select(num_cols)

col_names = corr_data.columns
features = corr_data.rdd.map(lambda row: row[0:])
corr_mat=Statistics.corr(features, method="pearson")
corr_df = pd.DataFrame(corr_mat)
corr_df.index, corr_df.columns = col_names, col_names

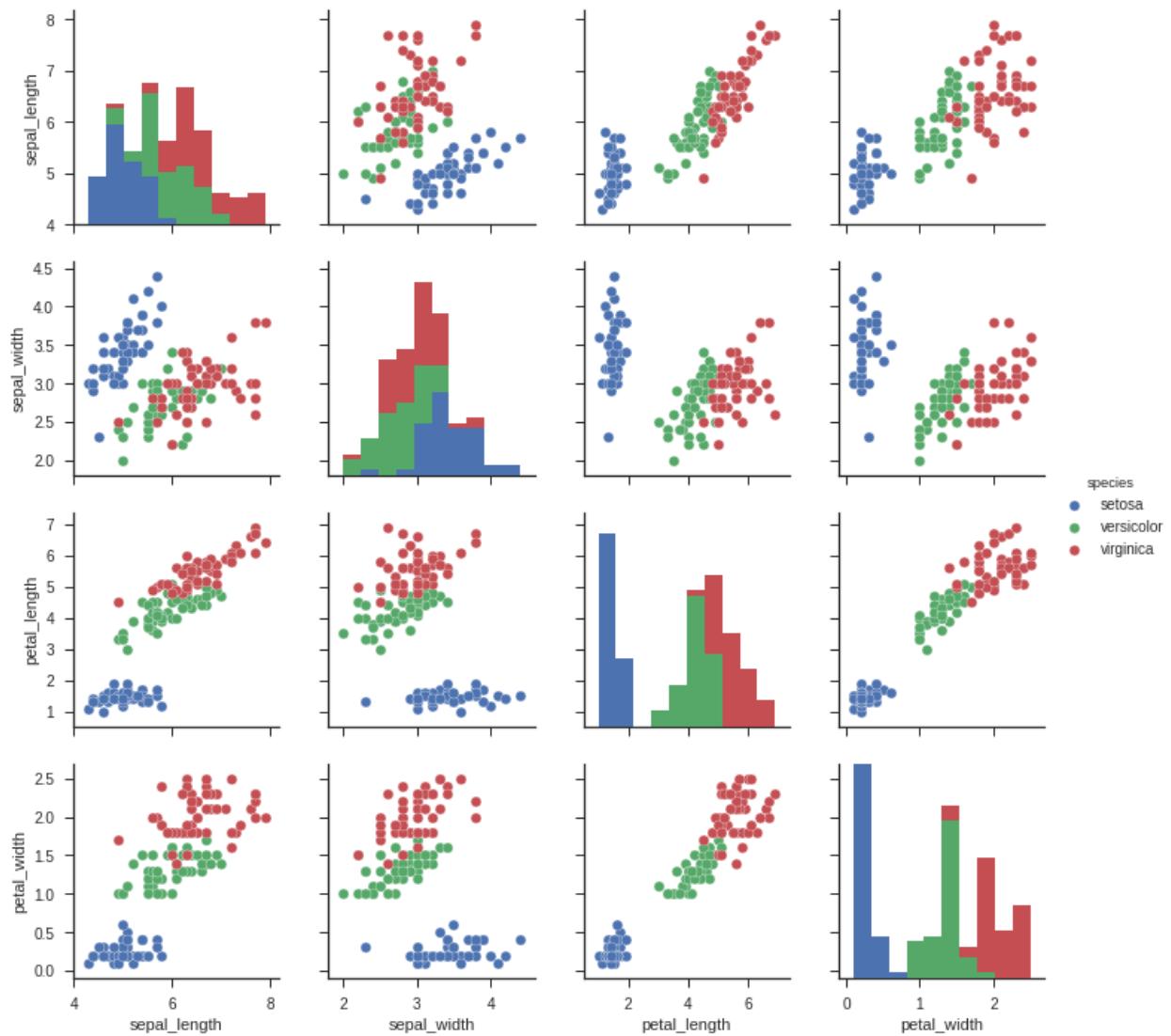
print(corr_df.to_string())
```

```
+-----+-----+
|      Account Balance|      No of dependents|
+-----+-----+
|              1.0|-0.01414542650320914|
|-0.01414542650320914|              1.0|
+-----+-----+
```

Scatter Plot

```
import seaborn as sns
sns.set(style="ticks")

df = sns.load_dataset("iris")
sns.pairplot(df, hue="species")
plt.show()
```



7.2.2 Categorical V.S. Categorical

Pearson's Chi-squared test

Warning: `pyspark.ml.stat` is only available in Spark 2.4.0.

```
from pyspark.ml.linalg import Vectors
from pyspark.ml.stat import ChiSquareTest

data = [(0.0, Vectors.dense(0.5, 10.0)),
        (0.0, Vectors.dense(1.5, 20.0)),
        (1.0, Vectors.dense(1.5, 30.0)),
        (0.0, Vectors.dense(3.5, 30.0)),
        (0.0, Vectors.dense(3.5, 40.0)),
        (1.0, Vectors.dense(3.5, 40.0))]
df = spark.createDataFrame(data, ["label", "features"])

r = ChiSquareTest.test(df, "features", "label").head()
print("pValues: " + str(r.pValues))
print("degreesOfFreedom: " + str(r.degreesOfFreedom))
print("statistics: " + str(r.statistics))
```

```
pValues: [0.687289278791, 0.682270330336]
degreesOfFreedom: [2, 3]
statistics: [0.75, 1.5]
```

Cross table

```
df.stat.crosstab("age_class", "Occupation").show()
```

```
+-----+-----+-----+-----+
|age_class_Occupation| 1| 2| 3| 4|
+-----+-----+-----+-----+
|          <25| 4| 34|108| 4|
|          55-64| 1| 15| 31| 9|
|          25-34| 7| 61|269| 60|
|          35-44| 4| 58|143| 49|
|          65+| 5| 3| 6| 9|
|          45-54| 1| 29| 73| 17|
+-----+-----+-----+-----+
```

Stacked plot

```
labels = ['missing', '<25', '25-34', '35-44', '45-54', '55-64', '65+']
missing = np.array([0.000095, 0.024830, 0.028665, 0.029477, 0.031918, 0.037073,
    ↪0.026699])
man = np.array([0.000147, 0.036311, 0.038684, 0.044761, 0.051269, 0.059542, 0.
    ↪0.054259])
women = np.array([0.004035, 0.032935, 0.035351, 0.041778, 0.048437, 0.056236,
    ↪0.048091])
ind = [x for x, _ in enumerate(labels)]

plt.figure(figsize=(10,8))
plt.bar(ind, women, width=0.8, label='women', color='gold',
    ↪bottom=man+missing)
plt.bar(ind, man, width=0.8, label='man', color='silver', bottom=missing)
plt.bar(ind, missing, width=0.8, label='missing', color='#CD853F')

plt.xticks(ind, labels)
plt.ylabel("percentage")
plt.legend(loc="upper left")
plt.title("demo")

plt.show()
```

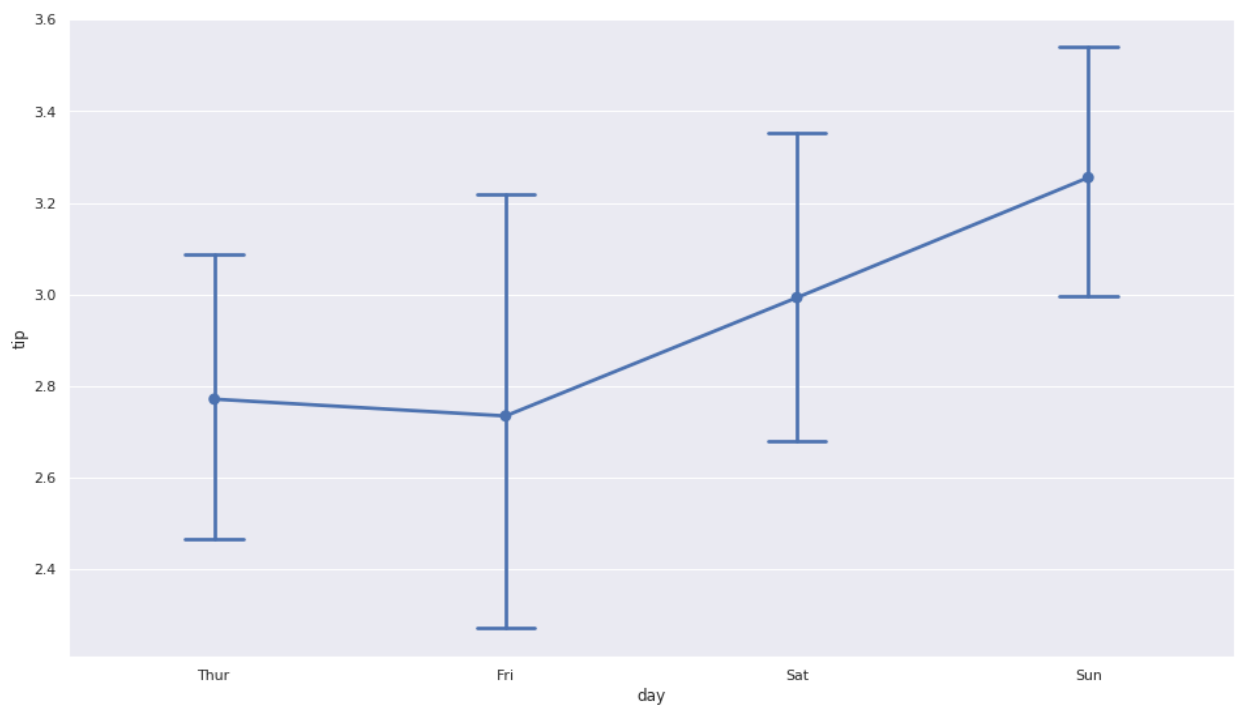
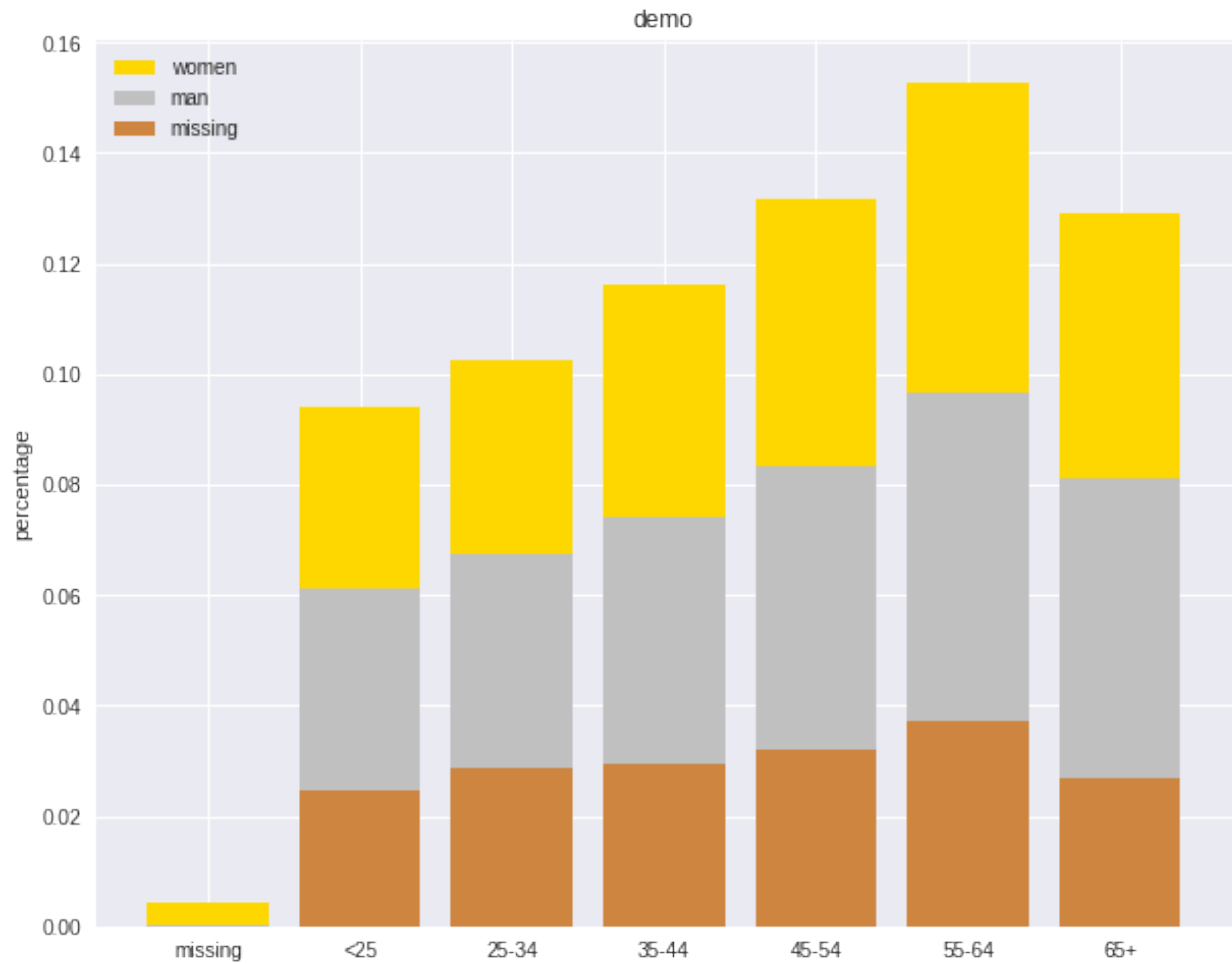
7.2.3 Numerical V.S. Categorical

Line Chart with Error Bars

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
%matplotlib inline

plt.rcParams['figure.figsize'] = (16,9)
plt.style.use('ggplot')
sns.set()

ax = sns.pointplot(x="day", y="tip", data=tips, capsize=.2)
plt.show()
```

Combination Chart

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
%matplotlib inline

plt.rcParams['figure.figsize'] =(16,9)
plt.style.use('ggplot')
sns.set()

#create list of months
Month = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'June',
        'July', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
#create list for made up average temperatures
Avg_Temp = [35, 45, 55, 65, 75, 85, 95, 100, 85, 65, 45, 35]
#create list for made up average percipitation %
Avg_Percipitation_Perc = [.90, .75, .55, .10, .35, .05, .05, .08, .20, .45, .
↪65, .80]
#assign lists to a value
data = {'Month': Month, 'Avg_Temp': Avg_Temp, 'Avg_Percipitation_Perc': Avg_
↪Percipitation_Perc}
#convert dictionary to a dataframe
df = pd.DataFrame(data)

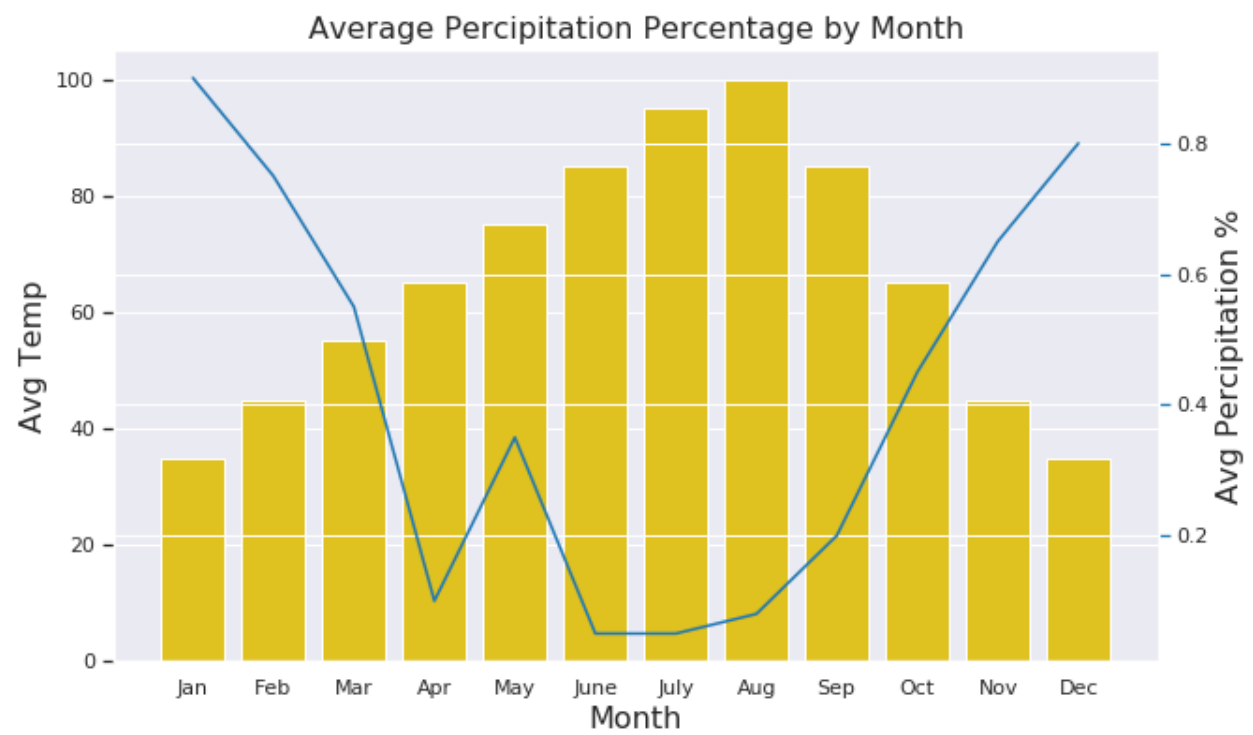
fig, ax1 = plt.subplots(figsize=(10,6))
ax1.set_title('Average Percipitation Percentage by Month', fontsize=16)
ax1.tick_params(axis='y')

ax2 = sns.barplot(x='Month', y='Avg_Temp', data = df, color = 'gold')
ax2 = ax1.twinx()
ax2 = sns.lineplot(x='Month', y='Avg_Percipitation_Perc', data = df,
↪sort=False, color='red')

ax1.set_xlabel('Month', fontsize=16)
ax1.set_ylabel('Avg Temp', fontsize=16)

ax2.tick_params(axis='y', color='red')
ax2.set_ylabel('Avg Percipitation %', fontsize=16)
plt.show()

```



DATA MANIPULATION: FEATURES

Chinese proverb

All things are difficult before they are easy!

Feature building is a super important step for modeling which will determine the success or failure of your model. Otherwise, you will get: garbage in; garbage out! The techniques have been covered in the following chapters, the followings are the brief summary. I recently found that the Spark official website did a really good job for tutorial documentation. The chapter is based on [Extracting transforming and selecting features](#).

8.1 Feature Extraction

8.1.1 TF-IDF

Term frequency-inverse document frequency (TF-IDF) is a feature vectorization method widely used in text mining to reflect the importance of a term to a document in the corpus. More details can be found at: <https://spark.apache.org/docs/latest/ml-features#feature-extractors>

[Stackoverflow](#) TF: Both HashingTF and CountVectorizer can be used to generate the term frequency vectors. A few important differences:

- a. partially **reversible** (CountVectorizer) vs **irreversible** (HashingTF) - since hashing is not reversible you cannot restore original input from a hash vector. From the other hand count vector with model (index) can be used to restore unordered input. As a consequence models created using hashed input can be much harder to interpret and monitor.
- b. **memory and computational overhead** - HashingTF requires only a single data scan and no additional memory beyond original input and vector. CountVectorizer requires additional scan over the data to build a model and additional memory to store vocabulary (index). In case of unigram language model it is usually not a problem but in case of higher n-grams it can be prohibitively expensive or not feasible.
- c. hashing **depends on** a size of the vector , hashing function and a document. Counting depends on a size of the vector, training corpus and a document.

- d. **a source of the information loss** - in case of HashingTF it is dimensionality reduction with possible collisions. CountVectorizer discards infrequent tokens. How it affects downstream models depends on a particular use case and data.

HashingTF and **CountVectorizer** are the two popular algorithms which used to generate term frequency vectors. They basically convert documents into a numerical representation which can be fed directly or with further processing into other algorithms like LDA, MinHash for Jaccard Distance, Cosine Distance.

- t : term
- d : document
- D : corpus
- $|D|$: the number of the elements in corpus
- $TF(t, d)$: Term Frequency: the number of times that term t appears in document d
- $DF(t, D)$: Document Frequency: the number of documents that contains term t
- $IDF(t, D)$: Inverse Document Frequency is a numerical measure of how much information a term provides

$$IDF(t, D) = \log \frac{|D| + 1}{DF(t, D) + 1}$$

- $TFIDF(t, d, D)$ the product of TF and IDF

$$TFIDF(t, d, D) = TF(t, d) \cdot IDF(t, D)$$

Let's look at the example:

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import HashingTF, IDF, Tokenizer

sentenceData = spark.createDataFrame([
    (0, "Python python Spark Spark"),
    (1, "Python SQL")],
    ["document", "sentence"])
```

```
sentenceData.show(truncate=False)
+-----+-----+
|document|sentence|
+-----+-----+
|0       |Python python Spark Spark|
|1       |Python SQL               |
+-----+-----+
```

Then:

- $TF(python, document1) = 1, TF(spark, document1) = 2$
- $DF(Spark, D) = 2, DF(sql, D) = 1$
- IDF:

$$IDF(\text{python}, D) = \log \frac{|D| + 1}{DF(t, D) + 1} = \log\left(\frac{2 + 1}{2 + 1}\right) = 0$$

$$IDF(\text{spark}, D) = \log \frac{|D| + 1}{DF(t, D) + 1} = \log\left(\frac{2 + 1}{1 + 1}\right) = 0.4054651081081644$$

$$IDF(\text{sql}, D) = \log \frac{|D| + 1}{DF(t, D) + 1} = \log\left(\frac{2 + 1}{1 + 1}\right) = 0.4054651081081644$$

- TFIDF

$$TFIDF(\text{python}, \text{document1}, D) = 3 * 0 = 0$$

$$TFIDF(\text{spark}, \text{document1}, D) = 2 * 0.4054651081081644 = 0.8109302162163288$$

$$TFIDF(\text{sql}, \text{document1}, D) = 1 * 0.4054651081081644 = 0.4054651081081644$$

Countvectorizer

[Stackoverflow TF](#): CountVectorizer and CountVectorizerModel aim to help convert a collection of text documents to vectors of token counts. When an a-priori dictionary is not available, CountVectorizer can be used as an Estimator to extract the vocabulary, and generates a CountVectorizerModel. The model produces sparse representations for the documents over the vocabulary, which can then be passed to other algorithms like LDA.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import CountVectorizer
from pyspark.ml.feature import HashingTF, IDF, Tokenizer

sentenceData = spark.createDataFrame([
    (0, "Python python Spark Spark"),
    (1, "Python SQL")],
    ["document", "sentence"])

tokenizer = Tokenizer(inputCol="sentence", outputCol="words")
vectorizer = CountVectorizer(inputCol="words", outputCol="rawFeatures")

idf = IDF(inputCol="rawFeatures", outputCol="features")

pipeline = Pipeline(stages=[tokenizer, vectorizer, idf])

model = pipeline.fit(sentenceData)
```

```
import numpy as np

total_counts = model.transform(sentenceData) \
    .select('rawFeatures').rdd \
    .map(lambda row: row['rawFeatures'].toArray()) \
    .reduce(lambda x, y: [x[i] + y[i] for i in range(len(y))])

vocabList = model.stages[1].vocabulary
d = {'vocabList': vocabList, 'counts': total_counts}
```

(continues on next page)

(continued from previous page)

```
spark.createDataFrame(np.array(list(d.values())) .T.tolist(), list(d.keys())) .
    show()
```

```
counts = model.transform(sentenceData).select('rawFeatures').collect()
counts

[Row(rawFeatures=SparseVector(8, {0: 1.0, 1: 1.0, 2: 1.0})),
 Row(rawFeatures=SparseVector(8, {0: 1.0, 1: 1.0, 4: 1.0})),
 Row(rawFeatures=SparseVector(8, {0: 1.0, 3: 1.0, 5: 1.0, 6: 1.0, 7: 1.0}))]
```

```
+-----+-----+
|vocabList|counts|
+-----+-----+
|   python|   3.0|
|   spark|   2.0|
|    sql|   1.0|
+-----+-----+
```

```
model.transform(sentenceData).show(truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
|document|sentence|words|
+-----+-----+-----+-----+-----+-----+-----+-----+
|rawFeatures|features|
+-----+-----+-----+-----+-----+-----+-----+-----+
|0|Python python Spark Spark|[python, python, spark, spark]|(3,[0,1],
|[2.0,2.0])|(3,[0,1],[0.0,0.8109302162163288])|
|1|Python SQL|[python, sql]|(3,[0,2],
|[1.0,1.0])|(3,[0,2],[0.0,0.4054651081081644])|
+-----+-----+-----+-----+-----+-----+-----+-----+
|rawFeatures|features|
```

```
from pyspark.sql.types import ArrayType, StringType

def termsIdx2Term(vocabulary):
    def termsIdx2Term(termIndices):
        return [vocabulary[int(index)] for index in termIndices]
    return udf(termsIdx2Term, ArrayType(StringType()))

vectorizerModel = model.stages[1]
vocabList = vectorizerModel.vocabulary
vocabList
```

```
['python', 'spark', 'sql']
```

```
rawFeatures = model.transform(sentenceData).select('rawFeatures')
rawFeatures.show()
```

(continues on next page)

(continued from previous page)

```
+-----+
|      rawFeatures|
+-----+
|(3,[0,1],[2.0,2.0])|
|(3,[0,2],[1.0,1.0])|
+-----+
```

```
from pyspark.sql.functions import udf
import pyspark.sql.functions as F
from pyspark.sql.types import StringType, DoubleType, IntegerType

indices_udf = udf(lambda vector: vector.indices.toList(),
    ↳ArrayType(IntegerType()))
values_udf = udf(lambda vector: vector.toArray().toList(),
    ↳ArrayType(DoubleType()))

rawFeatures.withColumn('indices', indices_udf(F.col('rawFeatures')))\
    .withColumn('values', values_udf(F.col('rawFeatures')))\
    .withColumn("Terms", termsIdx2Term(vocabList)("indices")).show()
```

```
+-----+-----+-----+-----+
|      rawFeatures|indices|      values|      Terms|
+-----+-----+-----+-----+
|(3,[0,1],[2.0,2.0])| [0, 1]| [2.0, 2.0, 0.0]| [python, spark]|
|(3,[0,2],[1.0,1.0])| [0, 2]| [1.0, 0.0, 1.0]| [python, sql]|
+-----+-----+-----+-----+
```

HashingTF

[Stackoverflow](#) TF: HashingTF is a Transformer which takes sets of terms and converts those sets into fixed-length feature vectors. In text processing, a “set of terms” might be a bag of words. HashingTF utilizes the hashing trick. A raw feature is mapped into an index (term) by applying a hash function. The hash function used here is MurmurHash 3. Then term frequencies are calculated based on the mapped indices. This approach avoids the need to compute a global term-to-index map, which can be expensive for a large corpus, but it suffers from potential hash collisions, where different raw features may become the same term after hashing.

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import HashingTF, IDF, Tokenizer

sentenceData = spark.createDataFrame([
    (0, "Python python Spark Spark"),
    (1, "Python SQL")],
    ["document", "sentence"])

tokenizer = Tokenizer(inputCol="sentence", outputCol="words")
```

(continues on next page)

(continued from previous page)

```
vectorizer = HashingTF(inputCol="words", outputCol="rawFeatures",
↳ numFeatures=5)

idf = IDF(inputCol="rawFeatures", outputCol="features")

pipeline = Pipeline(stages=[tokenizer, vectorizer, idf])

model = pipeline.fit(sentenceData)
model.transform(sentenceData).show(truncate=False)
```

```
+-----+-----+-----+-----+
↳ +-----+-----+-----+-----+
|document|sentence                |words                |
↳ |rawFeatures          |features                |
+-----+-----+-----+-----+
↳ +-----+-----+-----+-----+
|0        |Python python Spark Spark|[python, python, spark, spark]|(5, [0, 4],
↳ [2.0, 2.0])|(5, [0, 4], [0.8109302162163288, 0.0])|
|1        |Python SQL                |[python, sql]          |(5, [1, 4],
↳ [1.0, 1.0])|(5, [1, 4], [0.4054651081081644, 0.0])|
+-----+-----+-----+-----+
↳ +-----+-----+-----+-----+
```

8.1.2 Word2Vec

Word Embeddings

Word2Vec is one of the popular method to implement the **Word Embeddings**. [Word embeddings](#) (The best tutorial I have read. The following word and images content are from Chris Bail, PhD Duke University. So the copyright belongs to Chris Bail, PhD Duke University.) gained fame in the world of automated text analysis when it was demonstrated that they could be used to identify analogies. Figure 1 illustrates the output of a word embedding model where individual words are plotted in three dimensional space generated by the model. By examining the adjacency of words in this space, word embedding models can complete analogies such as “Man is to woman as king is to queen.” If you’d like to explore what the output of a large word embedding model looks like in more detail, check out this fantastic visualization of most words in the English language that was produced using a word embedding model called GloVe.

The Context Window

Word embeddings are created by identifying the words that occur within something called a “Context Window.” The Figure below illustrates context windows of varied length for a single sentence. The context window is defined by a string of words before and after a focal or “center” word that will be used to train a word embedding model. Each center word and context words can be represented as a vector of numbers that describe the presence or absence of unique words within a dataset, which is perhaps why word embedding models are often described as “word vector” models, or “word2vec” models.

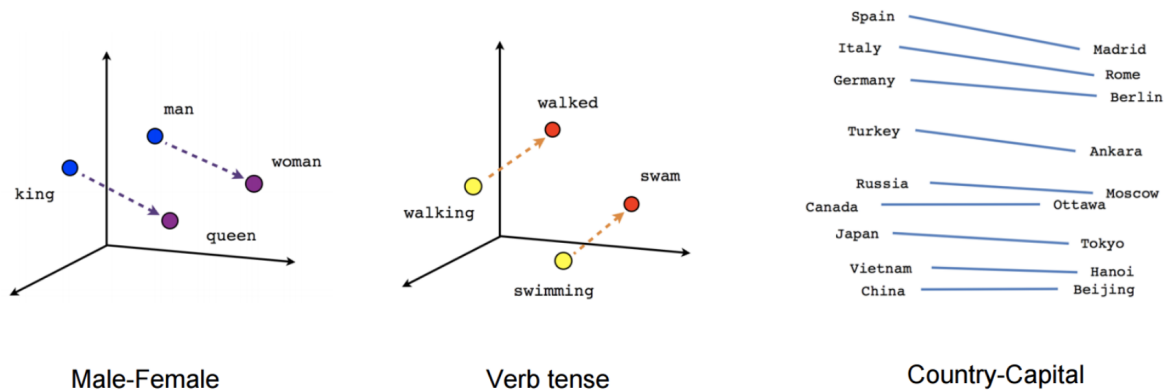


Fig. 1: output of a word embedding model

: Center Word
 : Context Word

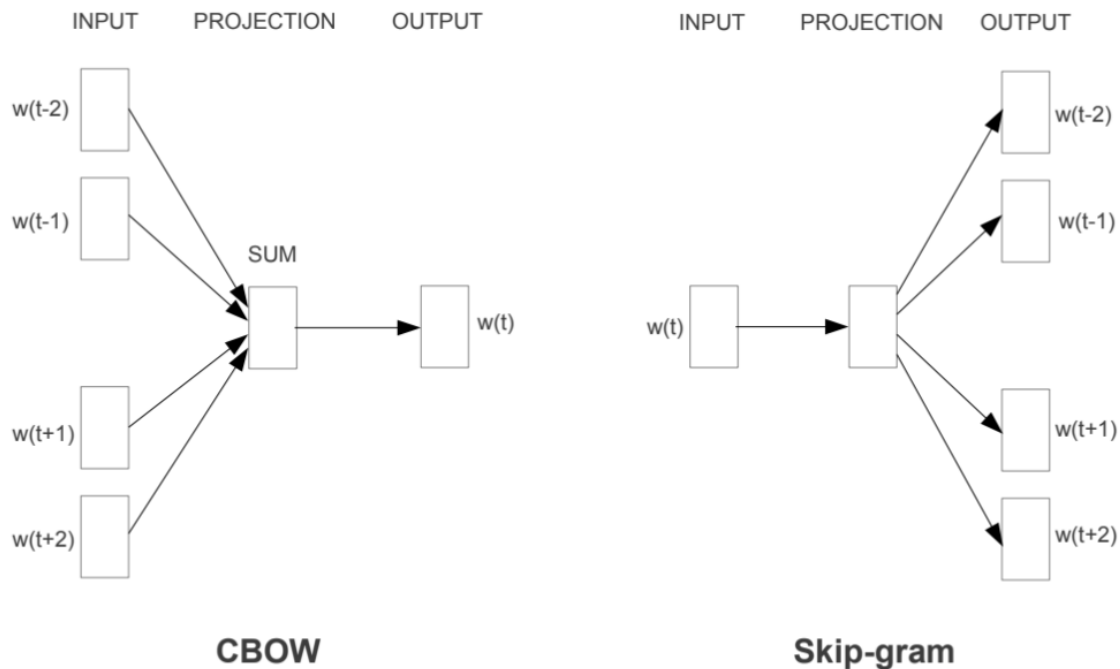
c=0 The cute cat jumps over the lazy dog.

c=1 The cute cat jumps over the lazy dog.

c=2 The cute cat jumps over the lazy dog.

Two Types of Embedding Models

Word embeddings are usually performed in one of two ways: “Continuous Bag of Words” (CBOW) or a “Skip-Gram Model.” The figure below illustrates the differences between the two models. The CBOW model reads in the context window words and tries to predict the most likely center word. The Skip-Gram Model predicts the context words given the center word. The examples above were created using the Skip-Gram model, which is perhaps most useful for people who want to identify patterns within texts to represent them in multidimensional space, whereas the CBOW model is more useful in practical applications such as predictive web search.



Word Embedding Models in PySpark

```
from pyspark.ml.feature import Word2Vec

from pyspark.ml import Pipeline

tokenizer = Tokenizer(inputCol="sentence", outputCol="words")
word2Vec = Word2Vec(vectorSize=3, minCount=0, inputCol="words", outputCol=
    ↪ "feature")

pipeline = Pipeline(stages=[tokenizer, word2Vec])

model = pipeline.fit(sentenceData)
result = model.transform(sentenceData)
```